Campus Smart-Parking System

Object-Oriented Analysis, Design and Java Implementation

Assignment Report: Tasks 1-12 with Complete-the-Code and Complexity Analyses

Item	Details
Student name / Register no.	B Hitesh Roy / 192372036
Course / Section	CSA0923 Programming with Java
GitHub repository	https://github.com/rootsolve12/campus-smart-parking
Language / tools	Java 21 (OpenJDK 21.0.10), Graphviz for diagrams, matplotlib for charts
Date of execution	23 October 2026
Total marks	50 (Tasks 1-11 = 42, Task 12 Design Challenge = 8)

Report map

Task	Topic	Marks	Section contains
1	Problem analysis and objects	4	IO analysis, candidate classes, rules, assumptions, domain diagram, parallel-array alternative
2	Data representation	3	Type choices, jagged array, casting-bug demo with output, 1-D / ArrayList alternative
3	Control logic + Complete the Code	4	Pseudocode, flowcharts, completed ParkingFacility with output, counters+stack alternative
4	Classes, objects, constructors	4	Class design, chaining, copy constructor, memory diagram, setter's/factory alternative
5	static members and life cycle	3	Counter design, GC demo, two-facility analysis
6	Inheritance hierarchy	4	UML, super, constructor trace, composition alternative
7	Overriding	4	Overriding rules, final, equals/hashCode, switch alternative
8	Polymorphism + Complete the Code	4	Overloading, dispatch trace, completed FeeDemo with output, instanceof alternative
9	Abstraction	4	Abstract class vs interface, reversed-roles alternative
10	Packages and encapsulation	3	Package diagram, access modifiers, wrapper classes, public-field alternative
11	Implement, test, defend	5	Full source, 34-case test table, hand-checked scenario, HashMap/ArrayList measurements
12	Day-4 Design Challenge	8	Impact analysis, evaluation, MODIFY decision, working code, evidence-based defence
A-C	Appendices	-	Repository layout, run commands, Git steps, supporting source listings

> **How the evidence in this report was produced.** Every program in this report was compiled and executed (Java 21). All outputs, test results and timings shown are copied from those real runs, not written by hand. Timings come from a single-CPU sandbox, so absolute nanoseconds will differ on your machine; the **growth trends (O-notation)** are what matter. Your own run will produce slightly different timing numbers, and you should state that in your viva.

TASK 1 - Understand the Problem and Identify the Objects (4 marks)

1.1 Inputs and outputs of the system

Event	Inputs received	Outputs produced
Set-up	Facility shape: number of floors and slots on each floor (e.g. 1x20, 3x50, 5x100, or irregular {3,2,4,1,2})	A facility in which every slot is free
Vehicle enters	Vehicle type (two-wheeler / car / EV / later e-bike, van), registration number, entry time	Allocated position {floor, slot}; parking ticket (unique number + entry time); or an error ("facility full", "duplicate vehicle", "unknown type")
Vehicle leaves	Registration number (to find the ticket), exit time, kWh used (EVs only), pass (monthly / one-day / none)	Slot released; receipt = parking fee + charging fee + total; or an error ("no active ticket", "exit before entry")
Management query	Report request	Occupied / capacity, total collection of the day

1.2 From nouns to classes, from verbs to methods

The problem statement was read sentence by sentence. **Nouns** that carry both data and behaviour became classes; nouns that are only values (fee, duration) became fields or local variables. **Verbs** became methods on the class that owns the data they change.

Candidate class	Source in the text	Responsibility	Why it is needed (justification)
`Vehicle` (abstract) with `TwoWheeler`, `Car`, `ElectricCar`	noun "vehicle", the three categories	Holds the registration number (identity); knows its hourly rate; computes its own fee (`calculateFee`)	Fees depend on the category. Putting the rate in the vehicle removes any "if type == ..." chain and lets new categories be added without editing old code (Task 12).
`ParkingFacility`	nouns "facility", "floor", "slot"	Owns the slot grid (`String[][]`); `allocate()` finds the first free slot; `release()` frees one; reports capacity and occupancy	Business rule "a slot cannot hold two vehicles" must be enforced in exactly one place, next to the data it protects.
`ParkingTicket`	noun "parking ticket"	Immutable record of one parking session: unique ticket number, vehicle, floor, slot, entry time	Links the entry event to the later exit event; "ticket number is never reused" is guaranteed by a single static counter here.
`ParkingService`	verbs "enters", "leaves", "allocate", "release", "compute"	Orchestrates use-cases `enter()` and `exit()`; keeps the map of active tickets; applies passes; accumulates the day's collection	Keeps `main()` thin and gives one class that coordinates the facility, tickets and pricing (a "facade").
`Pass` (`MonthlyPass`, `OneDayPass`)	phrases "monthly pass", "one-day pass", "visitors pay the full fee"	Answers one question: does this pass make parking free for this vehicle on this date?	Students/staff/visitors differ by what they hold, not by what they are. Modelling the pass keeps the user types out of the class tree.
`Receipt`	output "parking fee"	Immutable result of an exit: parking fee, charging fee, total	Lets `exit()` return two amounts without a tuple or two calls.

Nouns deliberately not turned into classes: *Floor* and *Slot* (they have no behaviour other than "occupied or free", so a jagged array is simpler and faster; they would become classes only if slots gained attributes such as "EV-charging" or "wide"); *Student / Staff / Visitor* (see `Pass` above); *fee* (a computed value).

Verb in the text	Becomes method	On class
enters / allocate a suitable free slot	`enter(type, regNo, time)`, `allocate(regNo)`	`ParkingService`, `ParkingFacility`
issue a ticket with a unique number	`new ParkingTicket(...)`, private `nextNo()`	`ParkingTicket`
record the entry time	constructor argument / `getEntryTime()`	`ParkingTicket`
leaves / release the slot	`exit(regNo, time, pass, kWh)`, `release(floor, slot)`	`ParkingService`, `ParkingFacility`
compute the parking fee	`calculateFee(minutes)`	`Vehicle` and subclasses
charged separately (EV charging)	`chargingFee(kWh)`	`Chargeable` (interface)
holding a valid pass park free	`covers(vehicle, entry)`	`Pass`

1.3 Domain model

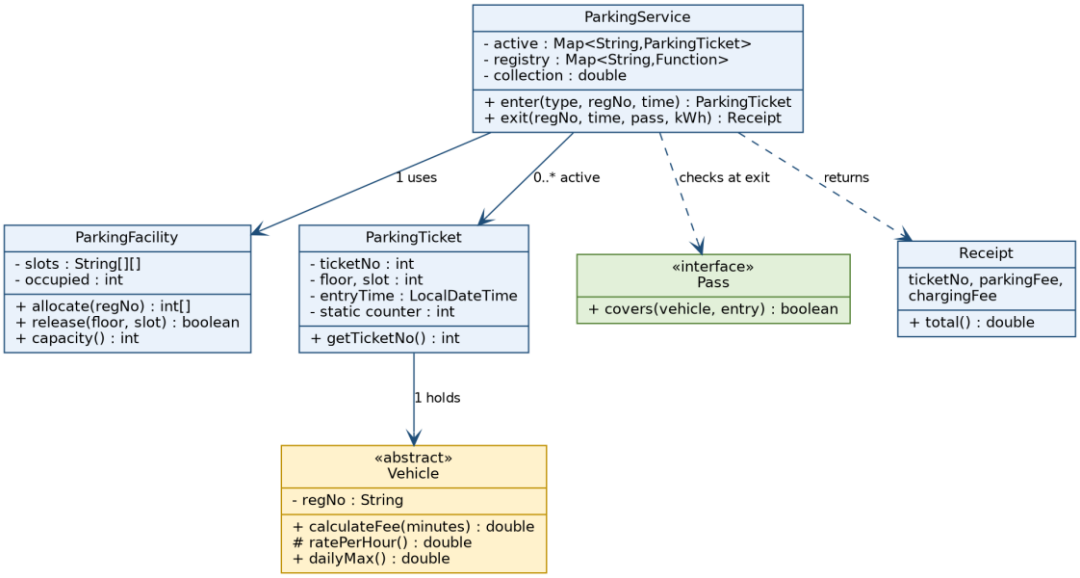


Figure 1.1 - Domain model: classes and their relationships

1.4 Business rules every valid operation must satisfy

#	Rule	Where it is enforced
BR1	A slot cannot hold two vehicles: allocation only writes into a `null` slot.	`ParkingFacility.allocate`
BR2	A registration number can be parked only once at a time (duplicate entry rejected).	`ParkingService.enter` (active map)
BR3	Ticket numbers strictly increase and are never reused (a failed entry does not consume a number because the ticket is created only after allocation succeeds).	`ParkingTicket` static counter
BR4	Entry time is fixed at issue; exit time may not be before entry time.	`ParkingTicket` (final field), `exit()`
BR5	A slot is released only when the position is valid and currently occupied; a released slot can be reused.	`ParkingFacility.release`
BR6	Every started hour is charged in full; the fee is capped per day; a zero-minute stay is free; duration cannot be negative.	`Vehicle.calculateFee`
BR7	A valid pass makes parking free; visitors without a pass pay the full fee; charging is always paid separately.	`ParkingService.exit`, `Pass`
BR8	Unknown vehicle type, blank registration number or negative duration is rejected with an exception.	`ParkingService.enter`, `Vehicle` constructor
BR9	When the facility is full, entry is refused and no ticket is issued.	`ParkingService.enter`
BR10	Only a `Chargeable` vehicle may be billed for charging.	`ParkingService.exit`

1.5 Assumptions about the facility

1. Any free slot suits any vehicle category. Zoning (two-wheeler bays, EV charging bays) is an extension: it would add a `slotType` to the grid.
2. Rates (Rs per hour): two-wheeler 10, car 30, electric car 30 plus Rs 12 per kWh for charging (these four values are given by the assignment). Daily maximum = 8 x hourly rate (my assumption; the text only says "daily maximum"). For Task 12: e-bike flat Rs 20 per day; delivery van Rs 30 off-peak and Rs 60 (double the car rate) from 09:00 to 17:00.
3. Registration numbers are case-insensitive and trimmed; they are normalised to upper case.
4. Time resolution is one minute. A stay longer than 24 h is billed as whole days at the daily maximum plus the remainder.
5. A van's hour is billed at the rate in force when that billed hour starts (08:30-09:30 starts off-peak; the next hour, starting 09:30, is peak).
6. A monthly pass is bound to one registration number and a date range; a one-day pass to one registration number and one calendar day (the day of entry).
7. The facility size is fixed when the object is created; the program is single-user (the ticket counter is `synchronized` anyway).
8. Money is in whole rupees; fees are stored as `double` because the given `calculateFee` signature returns `double` (all values here are whole numbers and therefore exact).

1.6 Alternative Concept and Complexity Analysis - vehicles as Strings in parallel arrays

What the solution would look like. There is no `Vehicle` class; three arrays are kept in step by a common index, and every operation is a static method that receives the arrays:

Parallel-array design (illustrative)

```
static String[] regNo     = new String[MAX];
static String[] type      = new String[MAX];   // "CAR", "TW", "EV"
static int[]    entryTime = new int[MAX];      // minutes since midnight
static int      count = 0;

static int find(String reg) {                  // linear search
    for (int i = 0; i < count; i++)
        if (regNo[i].equals(reg)) return i;
    return -1;
}
static int fee(int i, int exitTime) {          // behaviour lives OUTSIDE the data
    int hours = (exitTime - entryTime[i] + 59) / 60;
    switch (type[i]) {
        case "TW":  return hours * 10;
        case "CAR": return hours * 30;
        case "EV":  return hours * 30;
        default:    throw new IllegalStateException();
    }
}
// removing vehicle i must shift ALL THREE arrays, or the rows no longer match
```

Criterion	Vehicle objects (my design)	Parallel String/int arrays
Find by registration number	Linear scan `O(n)` (`O(1)` average if keyed in a `HashMap`)	Linear scan `O(n)` (same; can use a HashMap<String,Integer> to index)
Add a new attribute (e.g. `colour`)	Add one field + getter in one class; no other code changes: **O(1) places edited**	Add a fourth array and edit **every** insert, delete, shift and sort routine: **O(m) places edited**, risk of arrays going out of step
Behaviour (fee)	Inside the object, chosen by dynamic dispatch	External `switch` on a String: new type = edit the switch

Criterion	Vehicle objects (my design)	Parallel String/int arrays
Type safety	Compiler checks types	`"CAR"` vs `"Car"` typo compiles and fails at run time
Space per vehicle (excluding the shared regNo String)	28 B measured (24 B object + 4 B array slot)	8 B measured (4 B type reference + 4 B int)

Measured search time (worst case: the vehicle is the last one; best of 5 rounds):

n	objects
n=1000 (best of 5)	2.33 us
n=10000 (best of 5)	22.23 us
n=100000 (best of 5)	270.63 us

Space per vehicle: parallel arrays 8.0 B (n = 200,000, measured with the JVM heap before/after allocation).

> **Conclusion.** Search is `O(n)` in both designs and the measured times are equal within noise (the cost is dominated by `String.equals`). The object design costs about 20 extra bytes per vehicle, but it makes adding an attribute `O(1)` in code changes and removes the "arrays out of step" bug class. The 20 B is the price paid for safety and extensibility, and it is trivial for a campus facility (5 x 100 = 500 vehicles = about 10 KB).

TASK 2 - Choose the Data Representation (3 marks)

2.1 Data type chosen for each field

Field	Java type	Why this type
Floor number, slot number	`int`	Used as array indices, which Java requires to be `int`; values are small (<= 100).
Vehicle (registration) number	`String`	Contains letters and digits (TN01AB1111) and is never used in arithmetic; stored upper-case and trimmed.
Ticket number	`int` (static counter starting at 1000)	Whole number, increasing by 1, 2.1 billion tickets is enough; `long` would be used for a multi-year system.
Entry / exit time	`LocalDateTime`	Handles stays across midnight and across days, and gives the clock hour needed for the van's 09:00-17:00 rule. A plain `int` of minutes would lose the date.
Duration	`int` minutes	`Duration.between(...).toMinutes()` returns `long`; it is narrowed to `int` with `Math.toIntExact`, which throws instead of silently overflowing.
Billable hours	`int`	Computed as `(minutes + 59) / 60` (integer ceiling) so no floating-point rounding is involved.
Hourly rate, fee, charging fee	`double` (whole-rupee values)	The given `calculateFee` returns `double`. Whole numbers are exactly representable in binary. If paise were introduced the design would move to `long` paise or `BigDecimal` (see the 0.1 + 0.2 line in 2.3).
kWh used	`int`	Whole kWh billed; negative values rejected.
Slot state	`String` (regNo or `null`)	`null` means free, so no separate boolean array is needed and the occupant is known.

2.2 Representing floors and slots

A **jagged array** `String[][] slots` is used: `slots[f]` is a separate array whose length is the number of slots on floor `f`. A two-dimensional rectangular array (`new String[F][S]`) would force every floor to have the same size, which is false for the Task 12 facility ({3,2,4,1,2} slots). A one-dimensional array would need index arithmetic and offsets. The jagged array also mirrors the real building: "floor 2, slot 3" is `slots[2][3]`.

Jagged array construction

```
public ParkingFacility(int[] slotsPerFloor) {
    slots = new String[slotsPerFloor.length][];       // rows created first ...
    for (int f = 0; f < slotsPerFloor.length; f++)
        slots[f] = new String[slotsPerFloor[f]];      // ... each with its own length
}
```

2.3 Where casting / type conversion can produce a wrong fee

The fee for 130 minutes (2 h 10 min) of car parking must be **Rs 90** (three started hours at Rs 30). Four natural-looking expressions give **Rs 60**, a loss of Rs 30, because the remainder is destroyed by integer division *before* any cast or `ceil` can rescue it.

task2/CastingDemo.java

```
public class CastingDemo {
    public static void main(String[] args) {
        int minutes = 130;      // 2 h 10 min  -> 3 started hours
        int rate = 30;

        int wrong1 = (minutes / 60) * rate;                    // integer division drops remainder
        double wrong2 = (double) (minutes / 60) * rate;        // cast AFTER the division
        double wrong3 = Math.ceil(minutes / 60) * rate;        // ceil of an int division
```

```
        int wrong4 = (int) (minutes / 60.0) * rate;            // truncating cast
        int right1 = ((minutes + 59) / 60) * rate;            // integer-only ceiling
        double right2 = Math.ceil(minutes / 60.0) * rate;     // cast operand BEFORE dividing

        System.out.println("minutes=" + minutes + ", rate=" + rate + "  (correct fee = 90)");
        System.out.println("wrong1 (minutes/60)*rate              = " + wrong1);
        System.out.println("wrong2 (double)(minutes/60)*rate      = " + wrong2);
        System.out.println("wrong3 Math.ceil(minutes/60)*rate     = " + wrong3);
        System.out.println("wrong4 (int)(minutes/60.0)*rate       = " + wrong4);
        System.out.println("right1 ((minutes+59)/60)*rate         = " + right1);
        System.out.println("right2 Math.ceil(minutes/60.0)*rate   = " + right2);

        long huge = 3_000_000_000L;
        System.out.println("(int) 3_000_000_000L                  = " + (int) huge);
        try { Math.toIntExact(huge); }
        catch (ArithmeticException e) { System.out.println("Math.toIntExact rejects it: " + e.getMessage());
}

        System.out.println("0.1 + 0.2 (double)                    = " + (0.1 + 0.2));
        System.out.println("10 + 20 (int rupees)                  = " + (10 + 20));
    }
}
```

Program output

```
minutes=130, rate=30  (correct fee = 90)
wrong1 (minutes/60)*rate              = 60
wrong2 (double)(minutes/60)*rate      = 60.0
wrong3 Math.ceil(minutes/60)*rate     = 60.0
wrong4 (int)(minutes/60.0)*rate       = 60
right1 ((minutes+59)/60)*rate         = 90
right2 Math.ceil(minutes/60.0)*rate   = 90.0
(int) 3_000_000_000L                  = -1294967296
Math.toIntExact rejects it: integer overflow
0.1 + 0.2 (double)                    = 0.30000000000000004
10 + 20 (int rupees)                  = 30
```

- **wrong1**: `130 / 60` is integer division = 2; the remaining 10 minutes vanish.
- **wrong2**: `(double)(minutes / 60)` casts the already-truncated 2; converting to `double` afterwards cannot bring 0.17 back.
- **wrong3**: `Math.ceil(minutes / 60)` calls `ceil` on the integer 2; it is still 2.
- **wrong4**: `(int)(minutes / 60.0)` divides correctly (2.1667) and then the **cast truncates** to 2.
- **How the design avoids it:** `(minutes + 59) / 60` is a pure-integer ceiling (130 + 59 = 189, 189 / 60 = 3). The alternative `Math.ceil(minutes / 60.0)` also works because the `double` division happens first. The design uses the integer form so there is no floating-point conversion at all.
- **Narrowing long to int:** `(int) 3_000_000_000L` silently becomes -1294967296; `Math.toIntExact` throws `ArithmeticException` instead, so the design uses it.

2.4 Reading input

Two mechanisms are used. **Command-line arguments** give the facility shape (`java parking.app.Cli 3,2,4,1,2`), because it is fixed once per run and is a natural start-up parameter. A **`Scanner` on `System.in`** reads the stream of ENTER / EXIT commands, because vehicle events arrive one after another and the number of events is unknown in advance; `Scanner` also allows an input file by redirection (`< input/sample_input.txt`). Arguments alone cannot express an open-ended event stream.

Output of: java parking.app.Cli "3,2,4,1,2" < input/sample_input.txt

```
ENTER  Ticket#1001 TwoWheeler[TN01A1] floor 0 slot 0 @ 2026-10-23T08:00
ENTER  Ticket#1002 Car[TN02B2] floor 0 slot 1 @ 2026-10-23T08:15
ENTER  Ticket#1003 ElectricCar[TN03C3] floor 0 slot 2 @ 2026-10-23T09:00
ENTER  Ticket#1004 Car[TN04D4] floor 1 slot 0 @ 2026-10-23T08:00
ENTER  Ticket#1005 EBike[TN05E5] floor 1 slot 1 @ 2026-10-23T10:00
ENTER  Ticket#1006 DeliveryVan[TN06F6] floor 2 slot 0 @ 2026-10-23T08:30
ERROR: Duplicate registration number: TN02B2
ERROR: Unknown vehicle type: BUS
REPORT occupied=6/12 collection=Rs 0.0
EXIT   TN01A1 ticket#1001 parking=Rs 20.0 charging=Rs 0.0 total=Rs 20.0
```

```
EXIT   TN02B2 ticket#1002 parking=Rs 120.0 charging=Rs 0.0 total=Rs 120.0
EXIT   TN03C3 ticket#1003 parking=Rs 30.0 charging=Rs 120.0 total=Rs 150.0
EXIT   TN04D4 ticket#1004 parking=Rs 0.0 charging=Rs 0.0 total=Rs 0.0
EXIT   TN05E5 ticket#1005 parking=Rs 20.0 charging=Rs 0.0 total=Rs 20.0
EXIT   TN06F6 ticket#1006 parking=Rs 90.0 charging=Rs 0.0 total=Rs 90.0
ERROR: No active ticket for TN77Q7
REPORT occupied=0/12 collection=Rs 400.0
```

2.5 Alternative Concept and Complexity Analysis - 1-D array or ArrayList<String[]>

One-dimensional array with `index = floor x slotsPerFloor + slot`. This formula assumes every floor has the same number of slots. With different sizes it breaks: floor 1 of {3,2,4,1,2} would map onto slots of floor 2. The fix is a **prefix-sum offsets array** `off[]`, where `off[f]` is the start index of floor `f`:

```
1-D with offsets
int[] off = new int[F + 1];                  // off[0] = 0
for (int f = 0; f < F; f++) off[f + 1] = off[f] + size[f];
String[] flat = new String[off[F]];          // total slots

String get(int f, int s)  { return flat[off[f] + s]; }      // O(1)
int floorOf(int g) {                                         // global index -> floor
    int p = java.util.Arrays.binarySearch(off, g);
    return p >= 0 ? p : -p - 2;                              // O(log F)
}
```

`ArrayList<String[]>`: each element is one floor's array. `floors.get(f)[s]` is `O(1)`, and floors can be added at run time (`floors.add(new String[n])`), which a fixed jagged array cannot do without copying.

Operation / property	Jagged `String[][]` (chosen)	1-D array + offsets	`ArrayList<String[]>`
Locate slot (floor, slot)	`O(1)`	`O(1)` (one addition)	`O(1)` (`get` + index)
Map global index to floor	`O(F)` scan	`O(log F)` binary search	`O(F)` scan
Different slots per floor	Natural	Needs the extra `off[]` array	Natural
Add a floor later	Copy outer array `O(F)`	Re-allocate whole array `O(N)`	Amortised `O(1)`
Space (F floors, N slots)	`O(N + F)`	`O(N + F)`	`O(N + F)` plus list overhead

Measured (100 floors of 20-99 slots, 6,056 slots, 5 million random look-ups):

Measurement	Value
Locate slot, ns per operation	jagged 1.42
Global index to floor by binary search, ns per operation	38.08
Space for the structure (4-byte references)	jagged: 26240 B

> **Conclusion.** All three locate a known slot in `O(1)`; the 1-D array saves about 1.6 KB of array headers (24,660 B vs 26,240 B) but pays an `O(log F)` search whenever a global index must be converted back to a floor, and it is harder to read. `ArrayList` is about 2.5x slower per access in the measurement because of the extra method call and indirection, but it lets floors be added dynamically. For a fixed facility, the jagged array is the simplest correct representation.

TASK 3 - Design the Control Logic (4 marks)

3.1 Allocating a free slot

- **Search order.** Floors are scanned from floor 0 upward; inside each floor, slots from 0 upward (nested `for` loops). The first free slot found is used, so low floors fill first (nearest to the gate).
- **When it stops.** The moment a `null` slot is found: the slot is written and `return {f, s}` leaves **both** loops at once. If both loops finish, every slot is occupied and the method returns `null`.
- **break / continue / return.** `return` ends the method and both loops without any flag variable, which is why it is used. A labelled `break` or a `continue` is the equivalent when the method must do work after the loop (shown below). `continue` skips an occupied slot without nesting the rest of the body inside an `else`.
- **No free slot.** `allocate` returns `null`; `ParkingService.enter` turns that into an `IllegalStateException("Facility full")` **before** a ticket is created, so no ticket number is consumed (BR3, BR9).

Equivalent form using continue and a labelled break

```
int[] result = null;
search:                                          // label on the OUTER loop
for (int f = 0; f < slots.length; f++) {
    for (int s = 0; s < slots[f].length; s++) {
        if (slots[f][s] != null) continue;       // occupied: skip to next slot
        slots[f][s] = vehicleNo;                 // first free slot: take it
        result = new int[]{f, s};
        break search;                            // leave BOTH loops
    }
}
return result;                                   // null when the facility is full
```

3.2 Computing the parking fee with if-else and switch

- `switch (type)` maps the category to its hourly rate (an unknown type reaches `default` and throws).
- `if-else if` expresses the **slabs**: zero stay -> 0; first hour -> one rate; 2 to 8 hours -> rate x hours; beyond 8 hours -> daily maximum.
- The hours are computed with the integer ceiling `(minutes + 59) / 60`.
- In the final object-oriented system the `switch` disappears: each subclass supplies its own rate and the `if` slabs become `Math.min(hours * rate, dailyMax())` in `Vehicle.calculateFee` (Tasks 7-8).

3.3 Pseudocode

Written in a structured, numbered form: ALGORITHM name, INPUT, OUTPUT, BEGIN/END, keywords in capitals, one statement per numbered line, and indentation for nesting. (Check the keyword conventions against the SIMATS standard handout before submission; the structure below is the part that the standard asks for.)

Pseudocode 1 - Slot allocation

```
ALGORITHM Allocate(vehicleNo)
INPUT   vehicleNo : text;  slots[0..F-1][0..S(f)-1] : table of text or EMPTY
OUTPUT  pair (floor, slot), or NONE when the facility is full
BEGIN
 1.  FOR f <- 0 TO F - 1 DO
 2.      FOR s <- 0 TO LENGTH(slots[f]) - 1 DO
 3.          IF slots[f][s] = EMPTY THEN
 4.              slots[f][s] <- vehicleNo
 5.              RETURN (f, s)                // leaves both loops
 6.          END IF
 7.      END FOR
```

```
 8.  END FOR
 9.  RETURN NONE                                // every slot was occupied
END
```

Pseudocode 2 - Slot release

```
ALGORITHM Release(floor, slot)
INPUT   floor, slot : integers
OUTPUT  TRUE if a vehicle was removed, otherwise FALSE
BEGIN
 1.  IF floor < 0 OR floor >= F THEN RETURN FALSE
 2.  IF slot < 0 OR slot >= LENGTH(slots[floor]) THEN RETURN FALSE
 3.  IF slots[floor][slot] = EMPTY THEN RETURN FALSE      // already free
 4.  slots[floor][slot] <- EMPTY
 5.  RETURN TRUE
END
```

Pseudocode 3 - Fee computation

```
ALGORITHM ComputeFee(type, minutes)
INPUT   type : text;  minutes : integer
OUTPUT  fee : integer rupees
BEGIN
 1.  SELECT type
 2.      CASE "TW"  : rate <- 10
 3.      CASE "CAR" : rate <- 30
 4.      CASE "EV"  : rate <- 30
 5.      OTHERWISE  : ERROR "unknown vehicle type"
 6.  END SELECT
 7.  IF minutes < 0 THEN ERROR "negative duration"
 8.  hours <- (minutes + 59) DIV 60              // integer ceiling
 9.  IF hours = 0 THEN                          // slab 0
10.      RETURN 0
11.  ELSE IF hours = 1 THEN                     // slab 1: first hour
12.      RETURN rate
13.  ELSE IF hours <= 8 THEN                    // slab 2: each additional hour
14.      RETURN rate * hours
15.  ELSE                                       // slab 3: daily maximum
16.      RETURN rate * 8
17.  END IF
END
```

Pseudocode 4 - Exit processing

```
ALGORITHM ExitVehicle(regNo, exitTime, pass, kWh)
INPUT   regNo, exitTime, pass, kWh
OUTPUT  receipt (parkingFee, chargingFee, total)
BEGIN
 1.  ticket <- active[regNo];  IF ticket = NONE THEN ERROR "no active ticket"
 2.  IF exitTime < ticket.entryTime THEN ERROR "exit before entry"
 3.  minutes <- exitTime - ticket.entryTime;  days <- minutes DIV 1440
 4.  IF pass COVERS (ticket.vehicle, ticket.entryTime) THEN parking <- 0
 5.  ELSE parking <- days * dailyMax + ticket.vehicle.CalculateFee(entry clock, exit clock)
 6.  IF kWh > 0 THEN charging <- kWh * 12  ELSE charging <- 0
 7.  Release(ticket.floor, ticket.slot);  REMOVE active[regNo]
 8.  collection <- collection + parking + charging
 9.  RETURN receipt(ticket.number, parking, charging)
END
```

3.4 Flowcharts

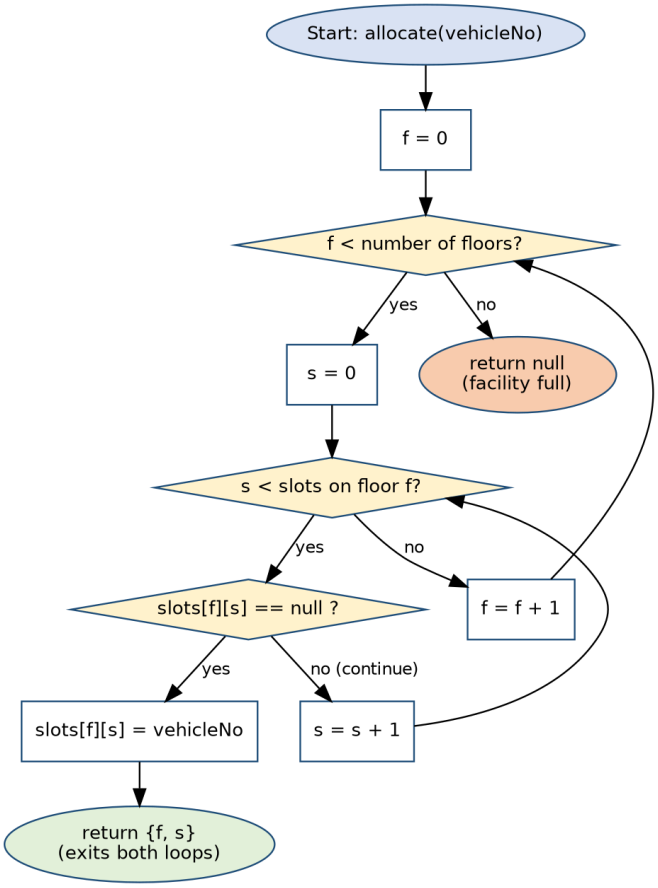


Figure 3.1 - Flowchart of allocate()

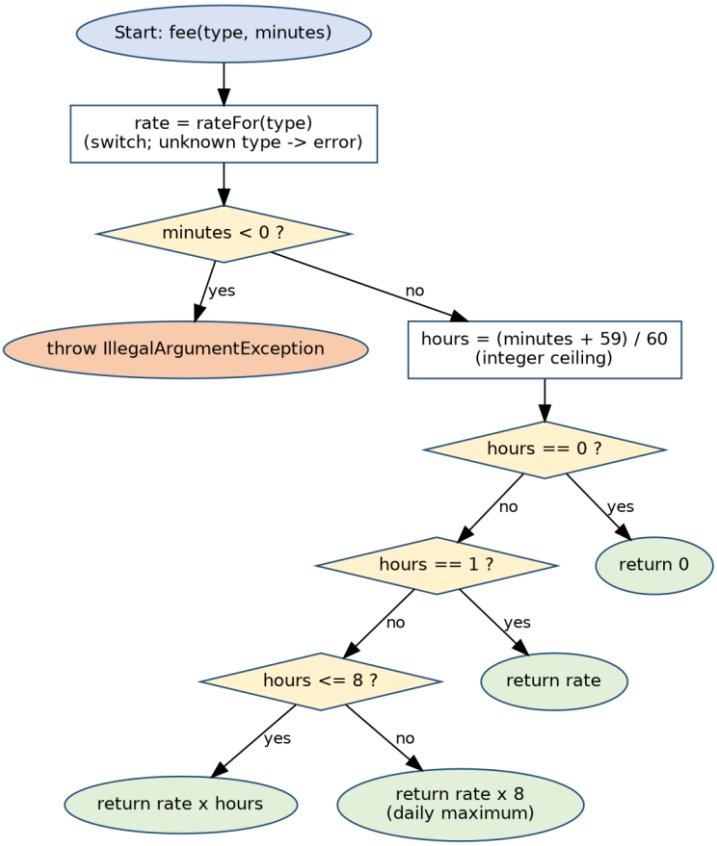


Figure 3.2 - Flowchart of the fee slabs

3.5 Complete the Code - ParkingFacility (completed and executed)

Both TODOs are filled in; `main()` and every other line are unchanged.

task3/ParkingFacility.java

```
public class ParkingFacility {
    // slots[floor][slot] holds a vehicle number, or null if the slot is free
    private final String[][] slots;

    public ParkingFacility(int[] slotsPerFloor) {
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            slots[f] = new String[slotsPerFloor[f]];
        }
    }

    /** Returns {floor, slot} of the first free slot, or null if it is full. */
    public int[] allocate(String vehicleNo) {
        // TODO 1 (completed): search floor by floor, slot by slot
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {          // first free slot found
                    slots[f][s] = vehicleNo;        // occupy it
                    return new int[]{f, s};         // return stops BOTH loops
                }
            }
        }
        return null;                                // every slot is occupied
    }

    /** Frees a slot. Returns false if the position is invalid or already free. */
    public boolean release(int floor, int slot) {
        // TODO 2 (completed): validate, free, return true; otherwise false
        if (floor < 0 || floor >= slots.length) return false;
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null;
        return true;
    }

    public static void main(String[] args) {
        // floor 0 has 2 slots, floor 1 has 1 slot
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});
        System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
                + p.release(5, 0));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

Expected output (from the assignment)

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

Actual output (java ParkingFacility): identical to the expected output

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

Why the output is correct. Floor 0 has two slots and floor 1 has one. The first three vehicles take `[0,0]`, `[0,1]`, `[1,0]`. The fourth finds no free slot and receives `null`. `release(0,1)` succeeds (`true`); the second

`release(0,1)` fails because the slot is already free (`false`); `release(5,0)` fails because floor 5 does not exist (`false`). The next allocation finds slot `[0,1]` free again.

Fee slabs with switch and if-else (executed)

task3/FeeSlabDemo.java

```
public class FeeSlabDemo {
    // switch: vehicle type -> hourly rate
    static int rateFor(String type) {
        switch (type) {
            case "TW":  return 10;
            case "CAR": return 30;
            case "EV":  return 30;
            default: throw new IllegalArgumentException("Unknown vehicle type: " + type);
        }
    }

    // if-else: fee slabs (zero stay, first hour, 2..8 hours, daily maximum)
    static int fee(String type, int minutes) {
        int rate = rateFor(type);
        if (minutes < 0) throw new IllegalArgumentException("Negative duration");
        int hours = (minutes + 59) / 60;          // ceiling using integers only
        if (hours == 0)       return 0;           // slab 0: zero-minute stay
        else if (hours == 1)  return rate;        // slab 1: first hour
        else if (hours <= 8)  return rate * hours;// slab 2: each additional hour
        else                  return rate * 8;    // slab 3: daily maximum
    }

    public static void main(String[] args) {
        int[] stays = {0, 45, 60, 61, 130, 480, 481, 700};
        System.out.println("minutes | TW | CAR | EV");
        for (int m : stays) {
            System.out.println(m + " | " + fee("TW", m) + " | " + fee("CAR", m) + " | " + fee("EV", m));
        }
        try { fee("BUS", 30); } catch (IllegalArgumentException e) { System.out.println("Rejected: " +
e.getMessage()); }
    }
}
```

Program output

```
minutes | TW | CAR | EV
0 | 0 | 0 | 0
45 | 10 | 30 | 30
60 | 10 | 30 | 30
61 | 20 | 60 | 60
130 | 30 | 90 | 90
480 | 80 | 240 | 240
481 | 80 | 240 | 240
700 | 80 | 240 | 240
Rejected: Unknown vehicle type: BUS
```

Row 61: 61 minutes is 2 started hours (Rs 20 / 60 / 60). Row 481: 9 started hours is capped at the daily maximum (80 / 240 / 240). Row 0 is free.

3.6 Alternative Concept and Complexity Analysis - free-slot count per floor + stack of free slots

What the logic becomes. Each floor keeps a counter `free[f]` and a stack `stack[f]` that holds its free slot numbers. Allocation skips full floors in `O(1)` each and pops a slot; release pushes the slot back.

Counters + stack

```
ALGORITHM AllocateFast(vehicleNo)
 1.  FOR f <- 0 TO F - 1 DO
 2.      IF free[f] > 0 THEN                       // skip full floors without looking inside
 3.          s <- POP(stack[f]);   free[f] <- free[f] - 1
 4.          slots[f][s] <- vehicleNo
 5.          RETURN (f, s)
 6.      END IF
 7.  END FOR
 8.  RETURN NONE

ALGORITHM ReleaseFast(floor, slot)
```

```
1.  validate floor and slot; IF slots[floor][slot] = EMPTY THEN RETURN FALSE
2.  slots[floor][slot] <- EMPTY
3.  PUSH(stack[floor], slot);   free[floor] <- free[floor] + 1
4.  RETURN TRUE
```

Java for the alternative (from bench/Benchmark.java)

```
boolean allocate(String v) {
    for (int f = 0; f < free.length; f++)
        if (free[f] > 0) {
            int s = stack[f][--top[f]];       // pop
            slots[f][s] = v; free[f]--;
            return true;
        }
    return false;
}
void release(int f, int s) { slots[f][s] = null; stack[f][top[f]++] = s; free[f]++; }
```

	Scan (my design)	Counters + stack
`allocate()` time	Worst case `O(F x S)` (all slots examined when nearly full)	`O(F)` (one counter test per floor, then `O(1)` pop); `O(1)` if one global stack is used
`release()` time	`O(1)`	`O(1)`
Extra space	`O(1)`	`O(F)` for counters and tops + `O(F x S)` for the stacks
Slot order	Always lowest free slot first (predictable)	Most recently freed slot is reused first (LIFO); a min-heap would restore ordering at `O(log S)`
Complexity of code	Very small	Three structures to keep consistent

Measured - filling an empty facility completely (average ns per `allocate()`):

Facility	scan
20 slots (1x20) avg ns per allocate()	15.1
150 slots (3x50) avg ns per allocate()	30.5
500 slots (5x100) avg ns per allocate()	97.0
10000 slots (20x500) avg ns per allocate()	3170.2
50000 slots (50x1000) avg ns per allocate()	8033.2

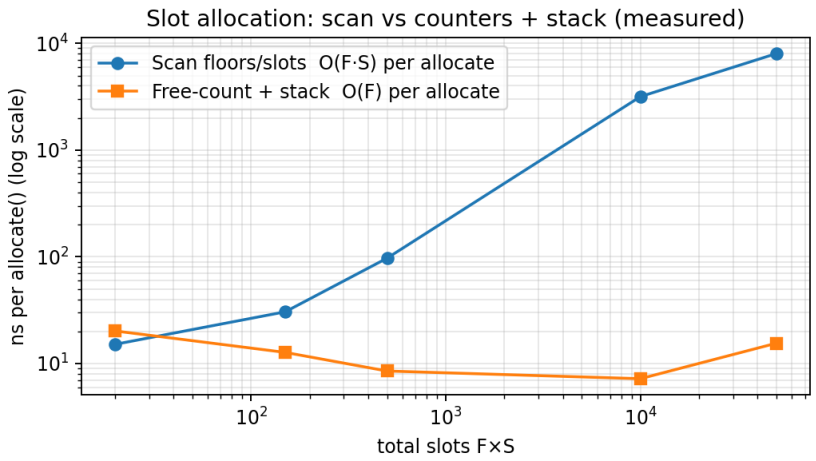


Figure 3.3 - Measured allocation cost

Conclusion. For the sizes in the scenario (20, 150 and 500 slots) both approaches take only tens of nanoseconds, so the simple scan is perfectly adequate and is used. The measurements follow the theory as the facility grows: the scan grows with F x S (about 3.2 us per allocation at 10,000 slots, 8 us at

> 50,000) while counters + stack stays at roughly 7-16 ns. If the campus facility grew to thousands of slots, the stack version would be worth its `O(F x S)` extra memory.

TASK 4 - Design Classes, Objects and Constructors (4 marks)

4.1 Fields and methods of each class

Class	Fields	Main methods
`Vehicle` (abstract)	`private final String regNo`; `static final int MINUTES_PER_DAY`	`protected Vehicle(String)`, `getRegNo()`, `abstract ratePerHour()`, `dailyMax()`, `calculateFee(int)`, `calculateFee(LocalTime, LocalTime)`, `equals`, `hashCode`, `toString`
`TwoWheeler`, `Car`, `ElectricCar`	none (rate is behaviour)	constructor with `super(regNo)`, `ratePerHour()`; `ElectricCar` adds `chargingFee(int kWh)`
`ParkingTicket`	`ticketNo`, `vehicle`, `floor`, `slot`, `entryTime`, `duplicate` (all `final`); `static int counter`	3 public constructors + 1 private, `getTicketNo()`, `issuedCount()` (static), `toString()`
`ParkingFacility`	`private final String[][] slots`; `private int occupied`	`allocate`, `release`, `floors`, `capacity`, `occupied`
`ParkingService`	`facility`, `active` (HashMap), `registry` (HashMap of factories), `collection`	`registerType`, `enter`, `exit`, `totalCollection`
`MonthlyPass`, `OneDayPass`	`regNo`, date(s)	`covers(Vehicle, LocalDateTime)`

4.2 Constructors and when each is used

- **Parameterised constructor** `ParkingTicket(Vehicle, floor, slot, entryTime)`: used by `ParkingService.enter` with the logical entry time; it asks the static counter for the next number.
- **Overloaded convenience constructor** `ParkingTicket(Vehicle, floor, slot)`: uses `LocalDateTime.now()`; it **chains** to the 4-argument constructor with `this(...)` so no code is repeated.
- **Private canonical constructor** (6 parameters): the single place where validation is written. Every other constructor ends up here (`this(...)` chaining), so there is one definition of a valid ticket.
- **Copy constructor** `ParkingTicket(ParkingTicket t)`: needed for **re-issuing a duplicate ticket** when a driver loses the original. It keeps the same ticket number and sets `duplicate = true`. A shallow copy is safe because every field is immutable (`String`, `int`, `LocalDateTime`) or a shared `Vehicle` reference that must stay shared.
- **No default (no-argument) constructor** in `ParkingTicket`: a ticket without a vehicle and a slot is meaningless. Declaring any constructor stops the compiler from generating the default one, which is what we want. `Vehicle` is abstract with a `protected` constructor so that only subclasses can call it.
- `this.field = parameter` is used wherever a parameter has the same name as the field it initialises.

parking/model/ParkingTicket.java

```
package parking.model;

import java.time.LocalDateTime;

public class ParkingTicket {
    private static int counter;                     // shared by ALL tickets / facilities
    static { counter = 1000; }                      // static block: one-time initialisation

    private final int ticketNo;
    private final Vehicle vehicle;
    private final int floor, slot;
    private final LocalDateTime entryTime;
    private final boolean duplicate;

    /** Convenience constructor: entry time = now. Chains to the 4-argument one. */
```

```
    public ParkingTicket(Vehicle vehicle, int floor, int slot) {
        this(vehicle, floor, slot, LocalDateTime.now());
    }
    /** Parameterised constructor: issues a NEW ticket number. */
    public ParkingTicket(Vehicle vehicle, int floor, int slot, LocalDateTime entryTime) {
        this(nextNo(), vehicle, floor, slot, entryTime, false);
    }
    /** Copy constructor: re-issues a DUPLICATE (same ticket number, flagged). */
    public ParkingTicket(ParkingTicket t) {
        this(t.ticketNo, t.vehicle, t.floor, t.slot, t.entryTime, true);
    }
    /** Single place where validation happens (all other constructors chain here). */
    private ParkingTicket(int no, Vehicle v, int floor, int slot, LocalDateTime entry, boolean dup) {
        if (v == null || entry == null) throw new IllegalArgumentException("vehicle and entry time
required");
        if (floor < 0 || slot < 0) throw new IllegalArgumentException("invalid floor/slot");
        this.ticketNo = no; this.vehicle = v; this.floor = floor; this.slot = slot;
        this.entryTime = entry; this.duplicate = dup;
    }

    private static synchronized int nextNo() { return ++counter; }
    public static int issuedCount() { return counter - 1000; }

    public int getTicketNo() { return ticketNo; }
    public Vehicle getVehicle() { return vehicle; }
    public int getFloor() { return floor; }
    public int getSlot() { return slot; }
    public LocalDateTime getEntryTime() { return entryTime; }
    public boolean isDuplicate() { return duplicate; }
    @Override public String toString() {
        return "Ticket#" + ticketNo + (duplicate ? "(dup)" : "") + " " + vehicle
                + " floor " + floor + " slot " + slot + " @ " + entryTime;
    }
}
```

4.3 How the objects are stored in memory

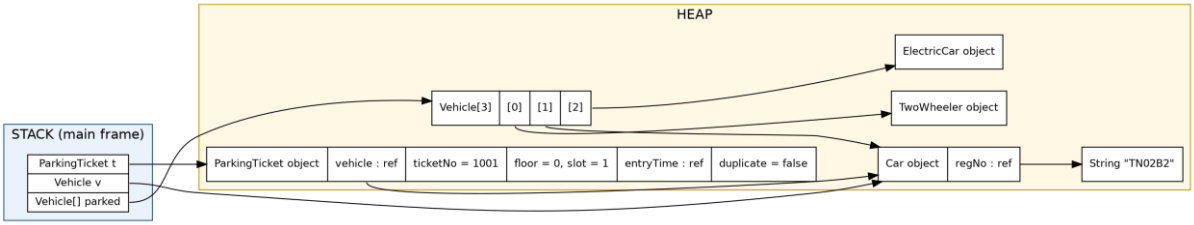


Figure 4.1 - Stack and heap for a ticket, a vehicle and a Vehicle[] array

- Reference variables (`t`, `v`, `parked`) live in the **stack frame** of the method that declares them; the objects they refer to live on the **heap**.
- A `ParkingTicket` object holds primitives (`ticketNo`, `floor`, `slot`, `duplicate`) inline and **references** to the `Vehicle` and `LocalDateTime` objects.
- The same `Car` object is shared by the ticket and by the `Vehicle[]` array element; there is one object and two references. No vehicle data is copied.
- `static int counter` is not in any object: it lives once per class (in the class area) and is shared by all tickets.
- An estimate for one ticket (compressed references): 12 B header + 4 + 4 + 4 + 4 + 4 + 1 = 33 B, rounded to 40 B, plus the shared `Vehicle` and the `LocalDateTime`.

4.4 Alternative Concept and Complexity Analysis - default constructor + setters, or static factory methods

The two alternatives

```
// (a) default constructor + setters
ParkingTicket t = new ParkingTicket();
t.setReg("TN01A1"); t.setFloor(1); t.setSlot(2); t.setEntryTime(now);
// the object exists - and is INVALID - between the first and last line
```

```
// (b) static factory methods with private constructor
public static ParkingTicket issue(Vehicle v, int floor, int slot, LocalDateTime when) { ... }
public static ParkingTicket reissue(ParkingTicket old)              { ... }
```

	Chained constructors (mine)	Default + setters	Static factory methods
Valid object guaranteed?	Yes: no object exists until validation passes	**No**: invalid between setter calls; a forgotten setter leaves a default (0/null) value	Yes: validation inside the factory
Immutability (`final` fields)	Possible	Impossible (fields must be non-final)	Possible
Self-describing names	No (overloads only)	n/a	`issue`, `reissue` explain intent
Can return a cached/sub-type object	No	No	Yes
Time to create n tickets	`O(n)`	`O(n)` (constant extra: 4 setter calls)	`O(n)`
Space for n tickets	`O(n)`	`O(n)`	`O(n)`

Measured creation time (n tickets, best of 5):

n	ctor chain
n=100000 tickets (ms, best of 5)	0.5
n=1000000 tickets (ms, best of 5)	18.1
n=5000000 tickets (ms, best of 5)	115.6

> **Conclusion.** All three are `O(1)` per object, `O(n)` for n objects, in both time and space; the measured differences are within about 10%, i.e. noise. The decision is therefore about **correctness**: constructor chaining (or factories) guarantees that every `ParkingTicket` that exists is valid and can be immutable; the setter style cannot. Factories would be the natural next step if named construction (`issue`, `reissue`) were wanted.

TASK 5 - Apply static Members and the Object Life Cycle (3 marks)

5.1 Where static is (and is not) used

Member	Why static	Why an instance member would be wrong
`ParkingTicket.counter` (`static int`)	One counter must be shared by every ticket so numbers never repeat (BR3).	Each ticket would have its own counter starting at 1000; every ticket would get 1001.
`static { counter = 1000; }` block	Runs once when the class is loaded: the initial value is set exactly once.	A constructor would reset it for each ticket.
`private static synchronized nextNo()`	Operates only on the static counter; `synchronized` makes the increment safe when two gates issue tickets together.	Needs no object; an instance method would have to be called on a ticket that does not exist yet.
`ParkingTicket.issuedCount()`	Class-level statistic (total tickets issued).	Meaningless for one ticket.
`Vehicle.MINUTES_PER_DAY`, `Chargeable.RATE_PER_KWH` (`static final`)	Constants shared by all vehicles; stored once.	A copy in every object wastes memory and can diverge.
`Pass.none()`, `Chargeable.isValidEnergy()` (interface static methods)	Utility behaviour that belongs to the type, not to an object.	-
`ParkingFacility.occupied` (instance)	**Not** static on purpose: each facility has its own occupancy. A static field would add the cars of the north and south gates together.	A static `campusOccupancy` is correct only for a campus-wide total.

5.2 What happens to a ParkingTicket after the vehicle leaves

- The ticket is created in `enter()` and stored in the `active` map (and referenced by the local variable returned to the caller).
- In `exit()` it is removed from the map and the slot is released. `Receipt` stores only the **int** ticket number, not the ticket object.
- When the caller's last reference goes out of scope, nothing refers to the ticket any more, so it becomes **eligible for garbage collection**. The garbage collector frees it at some later time of its choosing (calling `System.gc()` is only a request).

task5/StaticDemo.java

```
import java.lang.ref.WeakReference;

public class StaticDemo {
    static class InstanceCounterFacility {            // WRONG for two gates
        private int counter = 1000;
        int next() { return ++counter; }
    }
    static class StaticCounterFacility {              // ONE counter for the class
        private static int counter = 1000;
        int next() { return ++counter; }
    }
    static class PrefixedFacility {                   // instance counter + gate id
        private final String gate; private int counter = 1000;
        PrefixedFacility(String gate) { this.gate = gate; }
        String next() { return gate + "-" + (++counter); }
    }
    static class Ticket { final int no; Ticket(int no) { this.no = no; } }

    public static void main(String[] args) {
        InstanceCounterFacility n1 = new InstanceCounterFacility(), s1 = new InstanceCounterFacility();
        System.out.println("Instance counter : north " + n1.next() + "," + n1.next()
                + "  south " + s1.next() + "," + s1.next() + "   <-- duplicates!");
        StaticCounterFacility n2 = new StaticCounterFacility(), s2 = new StaticCounterFacility();
        System.out.println("Static counter   : north " + n2.next() + "," + n2.next()
                + "  south " + s2.next() + "," + s2.next() + "   <-- globally unique");
        PrefixedFacility n3 = new PrefixedFacility("N"), s3 = new PrefixedFacility("S");
        System.out.println("Prefixed counter : north " + n3.next() + "," + n3.next()
                + "  south " + s3.next() + "," + s3.next() + "   <-- unique per gate");
```

```
        Ticket t = new Ticket(1001);
        WeakReference<Ticket> weak = new WeakReference<>(t);
        System.out.println("Before exit, ticket reachable       : " + (weak.get() != null));
        t = null;                       // vehicle leaves, slot released, reference dropped
        for (int i = 0; i < 5 && weak.get() != null; i++) { System.gc(); }
        System.out.println("After last reference dropped, GC'd  : " + (weak.get() == null));
    }
}
```

Program output

```
Instance counter : north 1001,1002  south 1001,1002   <-- duplicates!
Static counter   : north 1001,1002  south 1003,1004   <-- globally unique
Prefixed counter : north N-1001,N-1002  south S-1001,S-1002   <-- unique per gate
Before exit, ticket reachable       : true
After last reference dropped, GC'd  : true
```

The last two lines use a `WeakReference` (which does not keep the object alive) to show that, once `t = null`, the ticket is collected.

5.3 How objects are passed to and returned from methods

- Java passes **everything by value**; for objects the value that is copied is the **reference**. `ParkingService.enter(...)` returns the `ParkingTicket` reference, so the caller and the `active` map refer to the **same** object.
- `exit(regNo, when, pass, kWh)` receives a `Pass` object. The method can call methods on it (`pass.covers(...)`) but assigning a new object to the parameter would not change the caller's variable.
- `new ParkingFacility(new int[]{3,2,4,1,2})`: the constructor reads the array but never stores it, so later changes to the caller's array cannot corrupt the facility (a defensive design).
- Primitives (`int minutes`, `int kWh`) are copied; changing them inside the method does not affect the caller.

5.4 Alternative Concept and Complexity Analysis - counter as an instance variable of ParkingFacility

If the counter were an instance variable and the university ran a north-gate and a south-gate facility, each object would start counting at 1000. The demo above shows **both gates issuing tickets 1001 and 1002**: duplicate ticket numbers, which violates BR3. Three possible solutions:

Solution	How	Space for the counter (k facilities, n tickets)	Comment
1. Keep it `static` (chosen)	One shared counter for all facilities	`O(1)`: one int	Globally unique numbers; one lock for all gates
2. Instance counter + gate prefix	`"N-1001"`, `"S-1001"` (ticket number becomes a String)	`O(k)`: one int per facility	Unique per gate; no cross-gate lock; the number type changes from int to String
3. Shared `AtomicInteger` / database sequence	One atomic counter passed to every facility	`O(1)`	Lock-free and safe with real concurrency; needs the sequence to be injected

The tickets themselves take `O(n)` space in all three. Only the counter differs: `O(1)` for solutions 1 and 3 and `O(k)` for solution 2.

TASK 6 - Design the Inheritance Hierarchy (4 marks)

6.1 Hierarchy and type of inheritance

`Vehicle` is the abstract superclass. `TwoWheeler` and `Car` extend it directly (**hierarchical** inheritance: one parent, several children) and `ElectricCar` extends `Car` (**multilevel**: Vehicle -> Car -> ElectricCar). The overall shape is therefore a **hybrid of hierarchical and multilevel** inheritance. Java allows only single class inheritance, so the extra capability "can be charged" is added through the **interface** `Chargeable`, not through a second parent.

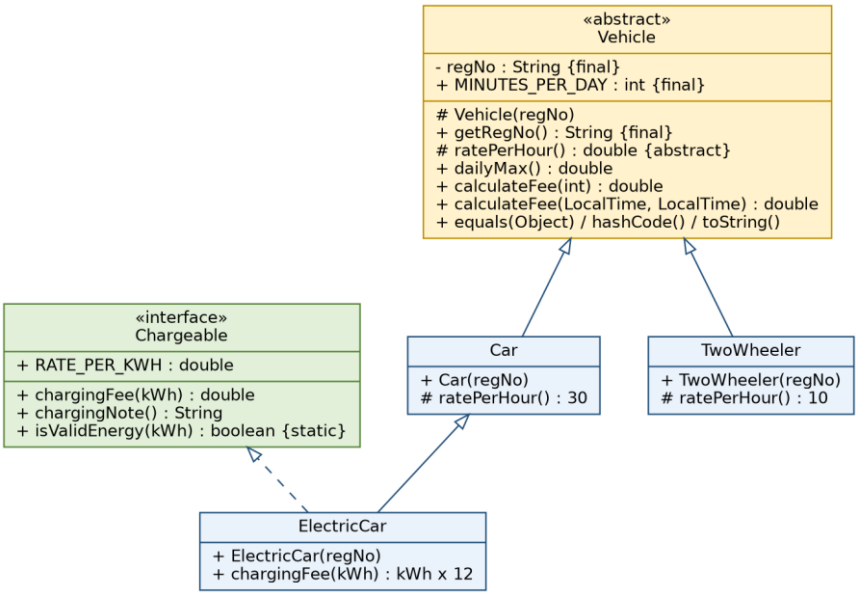


Figure 6.1 - UML class diagram of the original vehicle hierarchy

Users (student / staff / visitor) are not subclassed. A `Student` or `Staff` class would differ from `Visitor` only by "holds a pass", which is data, not behaviour. The `Pass` abstraction (Task 12) models that difference by composition and avoids a second hierarchy that would multiply with the vehicle hierarchy.

6.2 What is inherited and what each subclass adds

Class	Inherited from parent	Added / specialised
`TwoWheeler`	`regNo`, `getRegNo()`, `calculateFee(...)`, `dailyMax()`, `equals`, `hashCode`, `toString`	Implements abstract `ratePerHour()` = 10
`Car`	same as above	Implements `ratePerHour()` = 30
`ElectricCar`	Everything `Car` has (including rate 30)	Implements the `Chargeable` interface: `chargingFee(kWh)` = kWh x 12; no new rate

6.3 How super is used

- **Constructors:** `Car(String regNo) { super(regNo); }` hands the registration number up to `Vehicle`, which owns and validates it. `ElectricCar` calls `super(regNo)`, which reaches `Car`, which reaches `Vehicle`.
- **Fields:** `regNo` is `private` in `Vehicle`, so subclasses cannot write `super.regNo`; they use `getRegNo()`. This is deliberate encapsulation (Task 10).
- **Methods:** `super.method()` calls the parent's version of an overridden method. This design does not need it because every override replaces the behaviour entirely. Illustration: if `ElectricCar` wanted to extend the text, it could write `return super.toString() + " (EV)";`.

6.4 Constructor execution order when an ElectricCar is created

Rule: the first statement of every constructor is `super(...)` (explicit or implicit), so **parent constructors finish before child constructor bodies run**. The trace program below uses the same three-level structure with print statements:

task6/ConstructorTrace.java

```
class Vehicle {
    protected final String regNo;
    Vehicle(String regNo) { this.regNo = regNo; System.out.println("3. Vehicle(" + regNo + ") body runs
(Object() has already finished)"); }
}
class Car extends Vehicle {
    Car(String regNo) { super(regNo); System.out.println("4. Car(" + regNo + ") body runs"); }
}
class ElectricCar extends Car {
    private final int batteryKWh;
    ElectricCar(String regNo, int batteryKWh) {
        super(regNo);
        this.batteryKWh = batteryKWh;
        System.out.println("5. ElectricCar(" + regNo + ", " + batteryKWh + ") body runs");
    }
}
public class ConstructorTrace {
    public static void main(String[] args) {
        System.out.println("1. new ElectricCar(\"TN03C3\", 40) called -> ElectricCar constructor entered");
        System.out.println("2. its first statement super(...) pushes control up the chain");
        new ElectricCar("TN03C3", 40);
    }
}
```

Program output

```
1. new ElectricCar("TN03C3", 40) called -> ElectricCar constructor entered
2. its first statement super(...) pushes control up the chain
3. Vehicle(TN03C3) body runs   (Object() has already finished)
4. Car(TN03C3) body runs
5. ElectricCar(TN03C3, 40) body runs
```

Step	What happens	Stack of active constructors
1	`new ElectricCar("TN03C3", 40)`: the object is allocated on the heap, fields set to defaults, constructor entered	ElectricCar
2	First statement `super(regNo)` calls `Car(String)`	ElectricCar -> Car
3	`Car`'s first statement `super(regNo)` calls `Vehicle(String)`; `Vehicle` itself first calls `Object()` implicitly	ElectricCar -> Car -> Vehicle -> Object
4	`Object()` finishes; `Vehicle` body runs (stores `regNo`) -> prints line 3	ElectricCar -> Car -> Vehicle
5	`Vehicle` returns; `Car` body runs -> prints line 4	ElectricCar -> Car
6	`Car` returns; `ElectricCar` body runs (stores `batteryKWh`) -> prints line 5	ElectricCar

6.5 Alternative Concept and Complexity Analysis - composition (Vehicle has-a VehicleType)

Composition design

```
final class VehicleType {                       // data object: one per category, shared
    private final String name; private final double ratePerHour;
    VehicleType(String name, double rate) { this.name = name; this.ratePerHour = rate; }
    double rate() { return ratePerHour; }
}
final class Vehicle {                           // ONE concrete class
    private final String regNo; private final VehicleType type;
    Vehicle(String regNo, VehicleType type) { this.regNo = regNo; this.type = type; }
    double calculateFee(int minutes) { return ((minutes + 59) / 60) * type.rate(); }
}
// usage
VehicleType CAR = new VehicleType("Car", 30);
Vehicle v = new Vehicle("TN02B2", CAR);
```

	Inheritance (mine)	**Composition**
Add a new type with only a different rate	Write a new subclass (about 5 lines)	Create one `VehicleType` object; **no new class**
Add a type with different behaviour (e-bike flat fee, van peak hours)	Subclass overrides the method	`VehicleType` must become polymorphic (a strategy interface with several implementations), so classes are needed again
Change a vehicle's category at run time	Impossible (an object's class never changes)	Possible (swap the `type` reference)
Time to compute a fee	`O(1)`: one virtual call to `ratePerHour()`	`O(1)`: one field read + one call to `rate()`
Space per vehicle object	16 B (12 B header + 4 B `regNo` ref)	24 B (12 B header + 4 + 4 B refs, padded to 8): +8 B; the `VehicleType` objects are shared: `O(k)` total for k types
Natural reading	"A car **is a** vehicle"	"A vehicle **has a** category"

> **Conclusion.** Both compute a fee in `O(1)`. Composition costs 8 more bytes per vehicle (4 KB extra for 500 vehicles) but wins when types differ **only by data**. Inheritance wins when types differ **in behaviour** (the van, the e-bike), and it makes `ElectricCar implements Chargeable` natural. Because Task 12 introduces behaviour differences, inheritance is retained; the realistic hybrid is inheritance for behaviour plus a shared pricing-rule object when many types share the same rule.

TASK 7 - Override and Specialise Behaviour (4 marks)

7.1 Which methods are overridden and why

Method	Overridden in	Why the subclass version is needed
`ratePerHour()` (abstract)	`TwoWheeler` (10), `Car` (30), `EBike`, `DeliveryVan` (30 off-peak)	Each category has its own rate; the parent cannot know it.
`calculateFee(int)`	`EBike`	Flat fee per started day instead of an hourly rate.
`calculateFee(LocalTime, LocalTime)`	`DeliveryVan`	The rate depends on the clock hour in which each billed hour starts.
`dailyMax()`	`EBike` (20), `DeliveryVan` (480)	Different cap per category.
`chargingFee(int)` (interface method)	`ElectricCar`, `EBike`	Implements the `Chargeable` contract: kWh x 12.
`equals`, `hashCode`, `toString`	`Vehicle`	Override `Object`'s identity-based versions so equality means "same registration number".

7.2 @Override and the rules of overriding

- `@Override` makes the **compiler check** that a method really overrides something; a misspelt name (`ratePerHr`) becomes a compile error instead of a new, never-called method.
- **Signature:** same name and same parameter types (otherwise it is overloading, not overriding).
- **Return type:** the same type or a subtype (covariant). `calculateFee` returns `double` everywhere.
- **Access:** may not be narrower than the parent. `ratePerHour()` is `protected` in `Vehicle` and `protected` in subclasses (could be widened to `public`, never reduced to `private`).
- `static`, `private` and `final` methods cannot be overridden; an overriding method may not throw broader checked exceptions.

7.3 Where final is used

Use	Example	Reason
`final` field	`private final String regNo` in `Vehicle`; all fields of `ParkingTicket`	Identity and issue data must never change after construction (and `regNo` is a `HashMap` key, so changing it would lose the entry).
`final` constant	`static final int MINUTES_PER_DAY = 1440`	A named constant that cannot be reassigned.
`final` method	`public final String getRegNo()`	No subclass may alter how identity is read.
`final` class	`EBike`, `DeliveryVan`	Leaf classes not designed for extension; prevents accidental further subclassing.

7.4 toString(), equals() and hashCode()

From parking/model/Vehicle.java

```
@Override public boolean equals(Object o) {
    return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);   // identity = regNo only
}
@Override public int hashCode() { return regNo.hashCode(); }              // equal objects, equal hash
@Override public String toString() { return getClass().getSimpleName() + "[" + regNo + "]"; }
```

- **Contract:** `equals` is reflexive, symmetric and transitive (all three hold because it compares one `String`); two equal objects must return the same `hashCode` (they do: both use `regNo.hashCode()`).

- **Why both:** if only `equals` were overridden, a `HashSet`/`HashMap` would still treat the two objects as different (different default hash codes).

Output of parking.app.Polymorphism (equals / hashCode section is at the end)

```
--- Compile-time polymorphism (overloading) ---
calculateFee(130)                          = 90.0
calculateFee(LocalTime 08:00, 10:10)       = 90.0
calculateFee(23:00 -> 01:00, past midnight) = 60.0
--- Upcasting: one Vehicle[] holds every category ---
--- Runtime polymorphism (dynamic method dispatch) ---
Declared  Actual       calculateFee(int) runs in   ratePerHour() runs in      fee(130 min)
Vehicle   TwoWheeler   Vehicle                     TwoWheeler                 30.0
Vehicle   Car          Vehicle                     Car                        90.0
Vehicle   ElectricCar  Vehicle                     Car                        90.0
Vehicle   EBike        EBike                       EBike                      20.0
Vehicle   DeliveryVan  Vehicle                     DeliveryVan                90.0
--- equals / hashCode ---
Car("tn02b2").equals(ElectricCar("TN02B2 ")) = true
hash codes equal                             = true
HashSet already contains the 'other' object  = true
toString                                     = Car[TN02B2] / ElectricCar[TN02B2]
```

The registration is normalised in the constructor, so `Car("tn02b2")` equals `ElectricCar("TN02B2 ")`, and the `HashSet` that contains one reports that it contains the other.

7.5 Alternative Concept and Complexity Analysis - one calculateFee() with a switch on a type field

Switch-based alternative

```
public double calculateFee(int minutes) {
    int hours = (minutes + 59) / 60;
    switch (type) {                                  // "type" is a String / enum field
        case TWO_WHEELER: return hours * 10;
        case CAR:         return hours * 30;
        case EV:          return hours * 30;
        case EBIKE:       return 20;                 // <- every new type edits THIS method
        case VAN:         return vanFee(hours);
        default: throw new IllegalStateException(type + " has no fee rule");
    }
}
```

	Overriding (mine)	**switch on a type field**
Time per fee calculation, k types	`O(1)`: one virtual dispatch regardless of k	`O(1)` for a compiled jump table (enum/int); `O(k)` comparisons for a String switch or an if-else chain
Classes changed when a new type is added	**0 existing classes edited**, 1 new class	**1 existing class edited** (`Vehicle`), plus every other `switch (type)` in the program (`dailyMax`, display, reports)
Forgetting to handle a new type	Compile error (abstract method not implemented)	Silently reaches `default` at run time
Testing	Each class tested in isolation	One large method tested for every branch

Measured per-call cost for k = 3 and k = 8 types is given in Task 8.5. The conclusion is the same as in the time column: performance is not the deciding factor; **how many existing, already-tested classes must be edited** is.

TASK 8 - Apply Polymorphism (4 marks)

8.1 Compile-time polymorphism (overloading)

`Vehicle` has two methods called `calculateFee` with different parameter lists: `calculateFee(int minutes)` and `calculateFee(LocalTime in, LocalTime out)`. The **compiler** chooses between them by the number and static types of the arguments, before the program runs: one `int` argument selects the first, two `LocalTime` arguments select the second. (The second one converts the clock times to minutes, treating an earlier exit time as "after midnight", and delegates to the first.)

8.2 Upcasting: one array for every vehicle type

Upcasting

```
Vehicle[] lot = { new TwoWheeler("TN01A1"), new Car("TN02B2"), new ElectricCar("TN03C3"),
                  new EBike("TN05E5"), new DeliveryVan("TN06F6") };   // each element is upcast to Vehicle
```

8.3 Runtime polymorphism: dynamic method dispatch

Output of parking.app.Polymorphism (dispatch table)

```
--- Compile-time polymorphism (overloading) ---
calculateFee(130)                          = 90.0
calculateFee(LocalTime 08:00, 10:10)       = 90.0
calculateFee(23:00 -> 01:00, past midnight) = 60.0
--- Upcasting: one Vehicle[] holds every category ---
--- Runtime polymorphism (dynamic method dispatch) ---
Declared  Actual       calculateFee(int) runs in   ratePerHour() runs in      fee(130 min)
Vehicle   TwoWheeler   Vehicle                     TwoWheeler                 30.0
Vehicle   Car          Vehicle                     Car                        90.0
Vehicle   ElectricCar  Vehicle                     Car                        90.0
Vehicle   EBike        EBike                       EBike                      20.0
Vehicle   DeliveryVan  Vehicle                     DeliveryVan                90.0
--- equals / hashCode ---
Car("tn02b2").equals(ElectricCar("TN02B2 ")) = true
hash codes equal                             = true
HashSet already contains the 'other' object  = true
toString                                     = Car[TN02B2] / ElectricCar[TN02B2]
```

Trace of one call: Vehicle v = new ElectricCar("TN03C3"); v.calculateFee(130);

Step	Phase	What happens
1	Compile time	Declared type of `v` is **Vehicle**. The compiler checks `Vehicle` has a `calculateFee(int)` (it does) and fixes the *signature*.
2	Run time	The JVM looks at the **actual** object type: **ElectricCar**. It searches `ElectricCar` for `calculateFee(int)`: not declared; then `Car`: not declared; then `Vehicle`: found. Method executed: **Vehicle.calculateFee(int)** (table column 3).
3	Inside the method	hours = (130 + 59) / 60 = 3. The call `ratePerHour()` is again a virtual call on `this`, whose actual type is **ElectricCar** -> not declared -> **Car.ratePerHour()** runs (column 4) and returns 30.
4	Result	min(3 x 30, 240) = **90.0**. For the `EBike` the search ends at step 2 in `EBike` itself (it overrides `calculateFee(int)`), returning the flat 20.0.

8.4 Why this avoids long if-else chains

The caller writes `v.calculateFee(...)` and never asks "what kind of vehicle is this?". Adding `EBike` and `DeliveryVan` in Task 12 required **no change** to any code that loops over vehicles, because the JVM picks the right method. An `if (v instanceof Car) ... else if (v instanceof TwoWheeler) ...` chain would have to be edited in every place that computes a fee.

8.5 Complete the Code - FeeDemo (completed and executed)

The three TODOs are completed; the rest of the program is unchanged. TODO 1 is a pure-integer ceiling, TODO 2 a constructor with `super` plus the `ratePerHour()` override, TODO 3 the charging fee at Rs 12 per kWh.

task8/FeeDemo.java

```
interface Chargeable {
    double chargingFee(int kWh);
}

abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String regNo) { this.regNo = regNo; }
    public String getRegNo() { return regNo; }

    protected abstract double ratePerHour();

    /** Every started hour is charged in full. */
    public double calculateFee(int minutes) {
        // TODO 1 (completed): ceiling of minutes/60 with integer arithmetic only
        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }

    @Override public boolean equals(Object o) {
        return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);
    }
    @Override public int hashCode() { return regNo.hashCode(); }
}

class TwoWheeler extends Vehicle {
    TwoWheeler(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 10; }
}

class Car extends Vehicle {
    // TODO 2 (completed)
    Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30; }
}

class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) { super(regNo); }
    // TODO 3 (completed)
    @Override public double chargingFee(int kWh) { return kWh * 12.0; }
}

public class FeeDemo {
    public static void main(String[] args) {
        Vehicle[] parked = { new TwoWheeler("TN01A1"),
                             new Car("TN02B2"),
                             new ElectricCar("TN03C3") };
        for (Vehicle v : parked) {
            double fee = v.calculateFee(130); // dynamic method dispatch
            if (v instanceof Chargeable c) fee += c.chargingFee(10);
            System.out.println(v.getClass().getSimpleName() + " " + v.getRegNo()
                    + " -> Rs " + fee);
        }
        System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));
    }
}
```

Expected output (from the assignment)

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

Actual output (java FeeDemo): identical to the expected output

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
```

```
true
```

Vehicle	Hours billed	Rate	Parking	Charging	Total
TwoWheeler TN01A1	(130+59)/60 = 3	10	30	-	**30.0**
Car TN02B2	3	30	90	-	**90.0**
ElectricCar TN03C3	3	30	90	10 kWh x 12 = 120	**210.0**

Why the last line prints `true`. `new Car("TN02B2").equals(new ElectricCar("TN02B2"))` calls `Vehicle.equals`, which tests `o instanceof Vehicle` (an `ElectricCar` is a `Vehicle`) and then compares only the `regNo` strings, which are equal. The runtime class (`Car` versus `ElectricCar`) is never compared.

Is that the correct behaviour for this system? Yes. A registration number identifies one physical vehicle, so two records with the same number must be treated as the same vehicle. That is exactly what BR2 needs: a second entry with an already-parked number must be rejected as a duplicate, even if someone mislabels it as an EV. The trade-off is that `equals` ignores the category, so it cannot detect a data-entry conflict ("this number was a car yesterday"); the registry would have to report that separately.

8.6 Alternative Concept and Complexity Analysis - instanceof checks and casts

instanceof alternative

```
double fee(Vehicle v, int minutes) {
    int hours = (minutes + 59) / 60;
    if (v instanceof TwoWheeler)       return hours * 10;
    else if (v instanceof ElectricCar) return hours * 30;   // must test subclass BEFORE Car
    else if (v instanceof Car)         return hours * 30;
    else if (v instanceof EBike)       return 20;
    else if (v instanceof DeliveryVan) return hours * 30;   // (peak rule needs extra code)
    else throw new IllegalArgumentException();
}
```

Measured per-call cost (k = 3 types uses only the first three types; k = 8 uses eight):

Types	dynamic dispatch
, k=3 types (ns)	14.52
, k=8 types (ns)	18.10

	Dynamic dispatch (mine)	instanceof chain
Time per call, k types	`O(1)`	Up to `k` type tests: `O(k)`
Code size	Each type's rule sits in its own class	One method that grows with every type; **the order of the tests matters** (`ElectricCar` before `Car`)
New type	Add a class	Edit every chain in the program
Casting	None	Cast after each test; a wrong cast gives `ClassCastException`

> **Conclusion.** In this micro-benchmark the `switch` on a pre-computed integer is fastest (about 9-12 ns), and dynamic dispatch is a few nanoseconds slower than both alternatives at k = 3 and k = 8, so **I do not claim that dispatch is faster**; the JIT already optimises all three and every variant costs only about 9-18 ns per call. The theoretical `O(1)` vs `O(k)` difference is invisible at k <= 8. The real benefit is maintainability: zero edits to existing code when a type is added, no order-dependent test chain and no unchecked casts.

TASK 9 - Introduce Abstraction (4 marks)

9.1 What is abstract and what is an interface

Element	Kind	Reason
`Vehicle`	**abstract class**	Has **state** (`regNo`), a **constructor** that validates it, and shared concrete behaviour (`calculateFee`, `equals`, `hashCode`). It must not be instantiated: "a vehicle that is no particular category" has no rate.
`Vehicle.ratePerHour()`	**abstract method**	Each subclass must supply its own rate; the compiler enforces it.
`Chargeable`	**interface**	A **capability** that unrelated types can have (`ElectricCar`, `EBike`); a class can implement several interfaces but extend only one class.
`Pass`	**interface**	A **role** with one question (`covers`), implemented by two unrelated passes; also lets a lambda be a pass (`Pass.none()`).

9.2 Why an abstract class in one place and an interface in another

- **"is-a" with shared state and code -> abstract class.** All vehicles share `regNo`, validation and the fee skeleton.
- **"can-do" capability across unrelated types -> interface.** Charging is not a property of every vehicle (a two-wheeler cannot charge) and `ElectricCar` already has a parent (`Car`), so it cannot also extend a `Charger` class.
- **Multiple behaviours:** `EBike extends Vehicle implements Chargeable` and, if needed, `ElectricCar extends Car implements Chargeable, Payable`, with one class combining several contracts.

9.3 Where default and static interface methods help

parking/model/Chargeable.java

```
package parking.model;

/** Capability interface: anything that can be charged at a charging slot. */
public interface Chargeable {
    double RATE_PER_KWH = 12.0;                      // interface constant (public static final)

    double chargingFee(int kWh);

    default String chargingNote() {                  // default method: shared behaviour
        return "Charging billed separately at Rs " + RATE_PER_KWH + " per kWh";
    }
    static boolean isValidEnergy(int kWh) {          // static method: utility on the interface
        return kWh >= 0;
    }
}
```

- **default method** `chargingNote()`: gives every implementer a ready-made behaviour; a new method can be added to the interface later without breaking existing implementers.
- **static method** `isValidEnergy(int)`: a helper that belongs to the concept, called as `Chargeable.isValidEnergy(kWh)`; also `Pass.none()` as a named factory.
- **constant** `RATE_PER_KWH` (implicitly `public static final`): one place to change the tariff.

9.4 Alternative Concept and Complexity Analysis - Vehicle as an interface, Chargeable as an abstract class

Reversed roles

```
interface Vehicle {                          // alternative
    String getRegNo();                       // every class must now implement this (no shared field!)
```

```
    double ratePerHour();
    default double calculateFee(int minutes) { return ((minutes + 59) / 60) * ratePerHour(); }
}
abstract class Chargeable {                 // alternative
    abstract double chargingFee(int kWh);
}
class ElectricCar extends Car, Chargeable { }   // COMPILE ERROR: a class cannot extend two classes
```

What changes	Effect
State	An interface cannot hold instance fields, so `regNo` cannot live in `Vehicle`; every class would declare its own `regNo` and `getRegNo()`: duplicated code and no single place for validation.
Constructors	Interfaces have none; the rule "regNo must not be blank" cannot be enforced centrally (it would have to be repeated or moved to a factory).
equals / hashCode	An interface (even with default methods) **cannot define defaults for `Object` methods**, so the identity rule cannot be inherited.
Multiple inheritance	A class can implement many interfaces: that part gets better (a `Vehicle` can also be `Payable`). But `Chargeable` as an abstract class would stop `ElectricCar` from extending both `Car` and `Chargeable`: one of the two relationships must be lost.
Time per method call	Both `invokevirtual` (class) and `invokeinterface` are `O(1)`; after JIT compilation the difference is negligible and was not measurable here.
Space per object	Each object still stores one `regNo` reference: 16 B per vehicle, the same as now; the cost moves from memory to duplicated source code.

> **Conclusion.** Abstract class for the shared identity-and-state "kind of thing" (`Vehicle`); interface for the optional capability or role (`Chargeable`, `Pass`). Swapping them removes constructors, state and `equals/hashCode` inheritance from `Vehicle` and blocks `ElectricCar` from combining `Car` with `Chargeable`.

TASK 10 - Enforce Encapsulation through Packages and Access Modifiers (3 marks)

10.1 Package structure

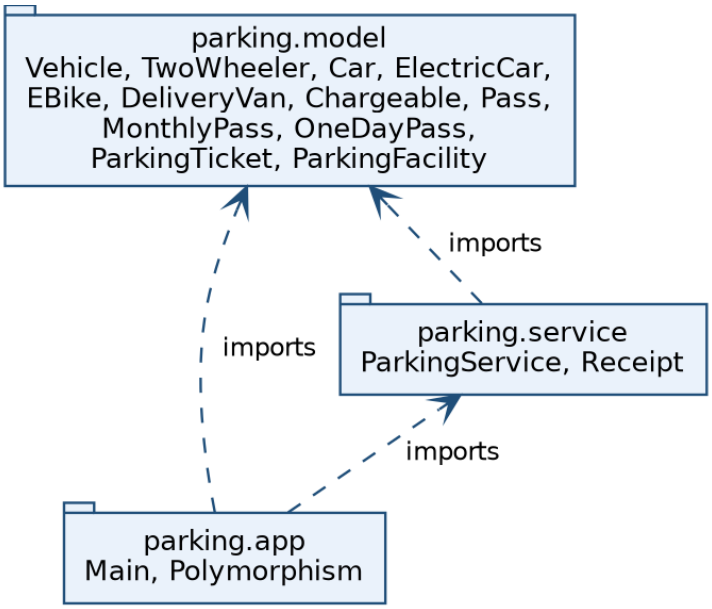


Figure 10.1 - Package structure and dependencies (arrows point to the package that is used)

Dependencies run **one way**: `app -> service -> model`. `model` knows nothing about the service or the console, so the domain classes can be reused with a GUI or a web front end without change.

10.2 Access modifier chosen for each member and why

Member	Modifier	Why
`Vehicle.regNo`, all `ParkingTicket` fields, `ParkingFacility.slots`/`occupied`, `ParkingService.active`/`collection`	`private`	Only the owning class may change state; others use methods that validate.
`Vehicle(String)` constructor	`protected`	Only subclasses may call it (the class is abstract anyway).
`Vehicle.ratePerHour()`	`protected abstract`	A hook for subclasses; callers outside the hierarchy must not need it.
`calculateFee`, `getRegNo`, `dailyMax`, `allocate`, `release`, `enter`, `exit`	`public`	The operations that other packages are meant to call (the API).
`ParkingTicket.nextNo()`	`private static synchronized`	Only constructors may take a number; nobody can burn or reset ticket numbers.
`ParkingTicket` canonical constructor	`private`	All validation is in one place and cannot be bypassed.
`Vehicle.minutesBetween` (static)	`protected`	Helper for subclasses (`DeliveryVan`) only.
(none in this design)	package-private (default)	Used when two classes in the same package must share a member but the world must not; no member needed it here.

10.3 Validation in getters, setters and constructors

- The objects are **immutable**, so there are no setters; validation sits in **constructors**, which is stronger than setter validation because an invalid object can never be built: blank `regNo` -> `IllegalArgumentException`; `floor < 0` or `slot < 0` -> exception; null vehicle or entry time ->

exception; `exit` before `entry` -> exception; negative `minutes` -> exception; a facility with a zero-slot floor -> exception.

- `ParkingFacility.release(floor, slot)` returns `false` for an invalid slot number instead of throwing, as the assignment's expected output requires.
- A validating setter would look like this (demo):

task10/Task10Demo.java
```
public class Task10Demo {
    static class Slot {
        private int number;
        void setNumber(int n) {
            if (n < 0) throw new IllegalArgumentException("Invalid slot number: " + n);
            this.number = n;
        }
        int getNumber() { return number; }
    }
    public static void main(String[] args) {
        Integer a = 127, b = 127, c = 128, d = 128;     // autoboxing
        System.out.println("127 == 127   (Integer) : " + (a == b));
        System.out.println("128 == 128   (Integer) : " + (c == d) + "   <-- compares references!");
        System.out.println("128.equals(128)        : " + c.equals(d));
        Slot s = new Slot();
        s.setNumber(7);
        System.out.println("valid setNumber(7)     : " + s.getNumber());
        try { s.setNumber(-3); } catch (IllegalArgumentException e) { System.out.println("setNumber(-3)
rejected : " + e.getMessage()); }
        Integer missing = null;
        try { int x = missing; } catch (NullPointerException e) { System.out.println("unboxing null         :
NullPointerException"); }
    }
}
```

Program output
```
127 == 127   (Integer) : true
128 == 128   (Integer) : false   <-- compares references!
128.equals(128)        : true
valid setNumber(7)     : 7
setNumber(-3) rejected : Invalid slot number: -3
unboxing null          : NullPointerException
```

10.4 Wrapper classes and autoboxing

- **Where used:** the collections hold wrapper-typed values (`Map<String, ParkingTicket>`, `HashSet<Vehicle>`); `Function<String, Vehicle>` boxes nothing, but `List<Integer>` would. Monetary and time arithmetic use primitives (`int`, `double`) on purpose.
- **Risk 1: `==` on wrappers compares references.** `Integer` values from -128 to 127 are cached, so `127 == 127` is `true` while `128 == 128` is `false` (see the output above). A ticket number such as 1001 compared with `==` would fail; always use `.equals()` or the primitive `int`.
- **Risk 2: unboxing `null`** throws `NullPointerException` (last line of the output): e.g. `int x = map.get("unknown")`.
- **Risk 3: performance:** boxing inside a loop allocates objects; primitives avoid that.

10.5 Alternative Concept and Complexity Analysis - public fields, validation by the caller

Public-field alternative
```
public class Slot { public int number; }           // alternative
// every caller must remember:
if (n < 0 || n >= facility.size()) throw new IllegalArgumentException();
slot.number = n;
```

- **What can go wrong:** (1) any caller can write `slot.number = -5` and nothing stops it; (2) the same check must be copied to **every place that updates the field**, so a forgotten check means a corrupt object; (3) a rule change (e.g. maximum slot 99) must be edited in all m places; (4) a mutable `regNo` that changes while the vehicle is a `HashMap` key makes the entry unreachable.

	Private + validating code (mine)	**Public fields**
Time per validated update	`O(1)`	`O(1)` per call-site (the same check, written by each caller)
Places where the check is written	**1**	**m** (one per caller)
Invalid state possible	No	Yes

Measured cost of validation (200 million updates; best of 5): validating setter 0.550 .

> **Conclusion.** The cost of the check is essentially zero (a fraction of a nanosecond, within noise of the unchecked store) and `O(1)` in both designs; the difference is that private fields keep it in **1 place instead of m** and make invalid states impossible.

TASK 11 - Implement, Test and Defend the Design (5 marks)

11.1 The implementation

The complete system is in three packages (see Task 10). The principal classes are listed in this report as follows: `Vehicle` and subclasses (Task 11.1 below), `ParkingTicket` (Task 4), `Chargeable` (Task 9), `EBike`, `DeliveryVan`, `Pass`, `OneDayPass` (Task 12); `Main`, `Cli` and `Polymorphism` are in Appendix B and `Benchmark` is in Appendix C. Everything is also in the GitHub repository.

parking/model/Vehicle.java

```
package parking.model;

import java.time.Duration;
import java.time.LocalTime;

/** Abstract superclass of every vehicle category. */
public abstract class Vehicle {
    public static final int MINUTES_PER_DAY = 1440;
    private final String regNo;                      // final: identity never changes

    protected Vehicle(String regNo) {
        if (regNo == null || regNo.isBlank())
            throw new IllegalArgumentException("Registration number is required");
        this.regNo = regNo.trim().toUpperCase();
    }

    public final String getRegNo() { return regNo; }

    /** Each concrete category supplies its own hourly rate. */
    protected abstract double ratePerHour();

    /** Maximum charge for one calendar day (8 hours at the hourly rate). */
    public double dailyMax() { return ratePerHour() * 8; }

    /** Every started hour is charged in full; capped at dailyMax() per day. */
    public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Negative duration: " + minutes);
        int days = minutes / MINUTES_PER_DAY;
        int rem = minutes % MINUTES_PER_DAY;
        int hours = (rem + 59) / 60;                 // integer ceiling - no precision loss
        return days * dailyMax() + Math.min(hours * ratePerHour(), dailyMax());
    }

    /** Overload: stay given as clock times; an exit "earlier" than entry means past midnight. */
    public double calculateFee(LocalTime in, LocalTime out) {
        return calculateFee(minutesBetween(in, out));
    }

    protected static int minutesBetween(LocalTime in, LocalTime out) {
        int m = (int) Duration.between(in, out).toMinutes();
        return m < 0 ? m + MINUTES_PER_DAY : m;
    }

    @Override public boolean equals(Object o) {
        return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);
    }
    @Override public int hashCode() { return regNo.hashCode(); }
    @Override public String toString() { return getClass().getSimpleName() + "[" + regNo + "]"; }
}
```

parking/model/TwoWheeler.java

```
package parking.model;
public class TwoWheeler extends Vehicle {
    public TwoWheeler(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 10; }
}
```

parking/model/Car.java

```
package parking.model;
```

```
public class Car extends Vehicle {
    public Car(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 30; }
}
```

parking/model/ElectricCar.java

```
package parking.model;
public class ElectricCar extends Car implements Chargeable {
    public ElectricCar(String regNo) { super(regNo); }
    @Override public double chargingFee(int kWh) {
        if (!Chargeable.isValidEnergy(kWh)) throw new IllegalArgumentException("Negative kWh");
        return kWh * RATE_PER_KWH;
    }
}
```

parking/model/ParkingFacility.java

```
package parking.model;

/** Jagged array of slots: floors may have different numbers of slots. */
public class ParkingFacility {
    private final String[][] slots;
    private int occupied;

    public ParkingFacility(int[] slotsPerFloor) {
        if (slotsPerFloor == null || slotsPerFloor.length == 0)
            throw new IllegalArgumentException("at least one floor required");
        slots = new String[slotsPerFloor.length][];
        for (int f = 0; f < slotsPerFloor.length; f++) {
            if (slotsPerFloor[f] <= 0) throw new IllegalArgumentException("floor " + f + " needs >= 1 slot");
            slots[f] = new String[slotsPerFloor[f]];
        }
    }

    public int[] allocate(String vehicleNo) {
        for (int f = 0; f < slots.length; f++) {
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) { slots[f][s] = vehicleNo; occupied++; return new int[]{f, s}; }
            }
        }
        return null;
    }
    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length) return false;
        if (slot < 0 || slot >= slots[floor].length) return false;
        if (slots[floor][slot] == null) return false;
        slots[floor][slot] = null; occupied--; return true;
    }
    public int floors() { return slots.length; }
    public int capacity() { int c = 0; for (String[] f : slots) c += f.length; return c; }
    public int occupied() { return occupied; }
}
```

parking/model/MonthlyPass.java

```
package parking.model;
import java.time.*;
/** Staff / student monthly pass: free parking while valid. */
public class MonthlyPass implements Pass {
    private final String regNo; private final LocalDate from, to;
    public MonthlyPass(String regNo, LocalDate from, LocalDate to) {
        if (to.isBefore(from)) throw new IllegalArgumentException("pass ends before it starts");
        this.regNo = regNo.trim().toUpperCase(); this.from = from; this.to = to;
    }
    @Override public boolean covers(Vehicle v, LocalDateTime entry) {
        LocalDate d = entry.toLocalDate();
        return v.getRegNo().equals(regNo) && !d.isBefore(from) && !d.isAfter(to);
    }
}
```

parking/service/ParkingService.java

```
package parking.service;

import parking.model.*;
import java.time.*;
```

```
import java.util.*;
import java.util.function.Function;

public class ParkingService {
    private final ParkingFacility facility;
    private final Map<String, ParkingTicket> active = new HashMap<>();          // regNo -> ticket
    private final Map<String, Function<String, Vehicle>> registry = new HashMap<>();
    private double collection;

    public ParkingService(ParkingFacility facility) { this.facility = facility; }

    /** Open/Closed: a new vehicle category is registered, this class is not edited. */
    public void registerType(String key, Function<String, Vehicle> factory) {
        registry.put(key.toUpperCase(), factory);
    }

    public ParkingTicket enter(String type, String regNo, LocalDateTime when) {
        Function<String, Vehicle> f = registry.get(type == null ? "" : type.toUpperCase());
        if (f == null) throw new IllegalArgumentException("Unknown vehicle type: " + type);
        Vehicle v = f.apply(regNo);
        if (active.containsKey(v.getRegNo()))
            throw new IllegalStateException("Duplicate registration number: " + v.getRegNo());
        int[] pos = facility.allocate(v.getRegNo());
        if (pos == null) throw new IllegalStateException("Facility full");
        ParkingTicket t = new ParkingTicket(v, pos[0], pos[1], when);
        active.put(v.getRegNo(), t);
        return t;
    }

    public Receipt exit(String regNo, LocalDateTime when, Pass pass, int kWh) {
        ParkingTicket t = active.get(regNo.trim().toUpperCase());
        if (t == null) throw new IllegalStateException("No active ticket for " + regNo);
        if (when.isBefore(t.getEntryTime())) throw new IllegalArgumentException("Exit before entry");
        Vehicle v = t.getVehicle();
        if (kWh > 0 && !(v instanceof Chargeable))
            throw new IllegalArgumentException(v + " cannot use a charging slot");

        long minutes = Duration.between(t.getEntryTime(), when).toMinutes();
        int fullDays = Math.toIntExact(minutes / Vehicle.MINUTES_PER_DAY);
        boolean free = pass != null && pass.covers(v, t.getEntryTime());
        double parking = free ? 0
                : fullDays * v.dailyMax()
                  + v.calculateFee(t.getEntryTime().toLocalTime(), when.toLocalTime());
        double charging = (kWh > 0 && v instanceof Chargeable c) ? c.chargingFee(kWh) : 0;

        facility.release(t.getFloor(), t.getSlot());
        active.remove(v.getRegNo());
        collection += parking + charging;
        return new Receipt(t.getTicketNo(), parking, charging);
    }

    public double totalCollection() { return collection; }
    public int activeCount() { return active.size(); }
    public ParkingFacility facility() { return facility; }
}
```

parking/service/Receipt.java

```
package parking.service;
public record Receipt(int ticketNo, double parkingFee, double chargingFee) {
    public double total() { return parkingFee + chargingFee; }
}
```

11.2 Scenario whose total is calculated by hand

Facility: 5 floors with {3, 2, 4, 1, 2} slots (12 slots). Date 23 October 2026. Seven vehicles enter and leave; the hand calculation was done **before** running the program.

Vehicle	Entry - Exit	Pass / kWh	Hand calculation	Hand (Rs)
TwoWheeler TN01A1	08:00 - 09:30	none	90 min -> 2 h x 10	20
Car TN02B2	08:15 - 11:16	none	181 min -> 4 h x 30	120

Vehicle	Entry - Exit	Pass / kWh	Hand calculation	Hand (Rs)
ElectricCar TN03C3	09:00 - 10:00	none, 10 kWh	60 min -> 1 h x 30 = 30; charging 10 x 12 = 120	150
Car TN04D4 (staff)	08:00 - 18:00	monthly pass	pass valid -> parking free	0
EBike TN05E5	10:00 - 10:20	none	flat fee	20
DeliveryVan TN06F6	08:30 - 10:30	none	08:30 hour off-peak 30 + 09:30 hour peak 60	90
Car TN07G7 (visitor)	11:00 - 13:00	one-day pass	pass valid -> free	0
Total collection			20+120+150+0+20+90+0	**400**

Program output (parking.app.Main - scenario part)

```
=== SCENARIO: facility 5 floors {3,2,4,1,2} = 12 slots, 23-Oct-2026 ===
ENTER  Ticket#1001 TwoWheeler[TN01A1] floor 0 slot 0 @ 2026-10-23T08:00
ENTER  Ticket#1002 Car[TN02B2] floor 0 slot 1 @ 2026-10-23T08:15
ENTER  Ticket#1003 ElectricCar[TN03C3] floor 0 slot 2 @ 2026-10-23T09:00
ENTER  Ticket#1004 Car[TN04D4] floor 1 slot 0 @ 2026-10-23T08:00
ENTER  Ticket#1005 EBike[TN05E5] floor 1 slot 1 @ 2026-10-23T10:00
ENTER  Ticket#1006 DeliveryVan[TN06F6] floor 2 slot 0 @ 2026-10-23T08:30
ENTER  Ticket#1007 Car[TN07G7] floor 2 slot 1 @ 2026-10-23T11:00
Occupied now: 7 / 12
EXIT   TN01A1 ticket#1001 parking=20.0 charging=0.0 total=20.0 (hand=20.0)
EXIT   TN02B2 ticket#1002 parking=120.0 charging=0.0 total=120.0 (hand=120.0)
EXIT   TN03C3 ticket#1003 parking=30.0 charging=120.0 total=150.0 (hand=150.0)
EXIT   TN04D4 ticket#1004 parking=0.0 charging=0.0 total=0.0 (hand=0.0)
EXIT   TN05E5 ticket#1005 parking=20.0 charging=0.0 total=20.0 (hand=20.0)
EXIT   TN06F6 ticket#1006 parking=90.0 charging=0.0 total=90.0 (hand=90.0)
EXIT   TN07G7 ticket#1007 parking=0.0 charging=0.0 total=0.0 (hand=0.0)
Hand-calculated collection = 400.0 | Program collection = 400.0 | match = true
```

> **Result:** hand-calculated collection **Rs 400.0** = program collection **Rs 400.0** (`match = true`); each of the seven receipts also equals its hand value.

11.3 Test table: expected versus actual

ID	Category	Test case	Expected	Actual	Result
T01	Normal	TwoWheeler, 130 min	30.0	30.0	PASS
T02	Normal	Car, 130 min	90.0	90.0	PASS
T03	Normal	ElectricCar 130 min + 10 kWh	210.0	210.0	PASS
T04	Normal	EBike, 20 min -> flat fee	20.0	20.0	PASS
T05	Normal	EBike, 7 h -> same flat fee	20.0	20.0	PASS
T06	Normal	Van 08:30-10:30 (1 off-peak h + 1 peak h)	90.0	90.0	PASS
T07	Normal	Van 10:00-12:00 (2 peak hours)	120.0	120.0	PASS
T08	Normal	Monthly pass covers valid regNo/date	true	true	PASS
T09	Normal	Monthly pass expired yesterday	false	false	PASS
T10	Boundary	Zero-minute stay (Car)	0.0	0.0	PASS
T11	Boundary	Exactly 60 min (Car) = 1 hour	30.0	30.0	PASS
T12	Boundary	61 min (Car) = 2 hours	60.0	60.0	PASS
T13	Boundary	Exactly 8 h (Car) = daily max	240.0	240.0	PASS
T14	Boundary	9 h (Car) capped at daily max	240.0	240.0	PASS
T15	Boundary	Van 16:30-17:30 = 1 h starting in peak	60.0	60.0	PASS
T16	Boundary	Van 16:30-17:31 (2nd hour starts 17:30)	90.0	90.0	PASS
T17	Boundary	Van 08:00-09:00 (peak starts at 09:00)	30.0	30.0	PASS
T18	Boundary	Car 23:00 -> 01:00 past midnight	60.0	60.0	PASS
T19	Boundary	Car exactly 24 h = one daily max	240.0	240.0	PASS
T20	Boundary	Last free slot used, next entry rejected	EXC IllegalStateException	EXC IllegalStateException	PASS
T21	Boundary	Slot is reused after the vehicle leaves	[0, 0]	[0, 0]	PASS

ID	Category	Test case	Expected	Actual	Result
T22	Invalid	Unknown vehicle type BUS	IllegalArgumentException	IllegalArgumentException	PASS
T23	Invalid	Duplicate registration number	IllegalStateException	IllegalStateException	PASS
T24	Invalid	Blank registration number	IllegalArgumentException	IllegalArgumentException	PASS
T25	Invalid	Negative duration	IllegalArgumentException	IllegalArgumentException	PASS
T26	Invalid	Exit of a vehicle that never entered	IllegalStateException	IllegalStateException	PASS
T27	Invalid	Exit time before entry time	IllegalArgumentException	IllegalArgumentException	PASS
T28	Invalid	Charging requested for a Car	IllegalArgumentException	IllegalArgumentException	PASS
T29	Invalid	release(0,0) valid then repeated	true false	true false	PASS
T30	Invalid	release(5,0) / release(0,-1) / release(1,9)	false false false	false false false	PASS
T31	Invalid	Facility with a zero-slot floor	IllegalArgumentException	IllegalArgumentException	PASS
T32	Contract	Car(TN02B2).equals(ElectricCar(tn02b2))	true	true	PASS
T33	Contract	Ticket numbers strictly increasing/unique	true	true	PASS
T34	Contract	Copy constructor keeps number, flags duplicate	true true	true true	PASS

SUMMARY: 34 tests, 34 passed, 0 failed. Coverage: normal operation (T01-T09), boundaries - last free slot, zero-minute stay, 60 and 61 minutes, daily-cap edge, peak-window edges, past-midnight, exactly 24 h (T10-T21), invalid inputs - unknown type, duplicate registration, invalid slot, negative duration, exit-before-entry, charging a car (T22-T31), object contracts (T32-T34).

A test finding worth reporting. In the first run, one test failed: I had expected Rs 90 for a van from 16:30 to 17:30. Re-deriving by hand showed that 16:30-17:30 is 60 minutes = **one** started hour, which starts in the peak window: Rs 60. The **program was right and my expected value was wrong**. I corrected the expectation and added the case 16:30-17:31 (two hours: 60 + 30 = 90), which makes the peak boundary at 17:00 visible. This is why expected values must be computed independently before running.

Sample run with an input file

input/sample_input.txt

```
# Campus Smart-Parking - sample input (date is fixed at 23-Oct-2026)
# ENTER <TYPE> <REG> <HH:mm>      EXIT <REG> <HH:mm> [kWh] [MONTHLY|ONEDAY]
ENTER TW    TN01A1 08:00
ENTER CAR   TN02B2 08:15
ENTER EV    TN03C3 09:00
ENTER CAR   TN04D4 08:00
ENTER EBIKE TN05E5 10:00
ENTER VAN   TN06F6 08:30
ENTER CAR   TN02B2 09:00
ENTER BUS   TN99Z9 09:10
REPORT
EXIT TN01A1 09:30
EXIT TN02B2 11:16
EXIT TN03C3 10:00 10
EXIT TN04D4 18:00 0 MONTHLY
EXIT TN05E5 10:20
EXIT TN06F6 10:30
EXIT TN77Q7 12:00
REPORT
```

Output of: java parking.app.Cli "3,2,4,1,2" < input/sample_input.txt

```
ENTER  Ticket#1001 TwoWheeler[TN01A1] floor 0 slot 0 @ 2026-10-23T08:00
ENTER  Ticket#1002 Car[TN02B2] floor 0 slot 1 @ 2026-10-23T08:15
ENTER  Ticket#1003 ElectricCar[TN03C3] floor 0 slot 2 @ 2026-10-23T09:00
ENTER  Ticket#1004 Car[TN04D4] floor 1 slot 0 @ 2026-10-23T08:00
ENTER  Ticket#1005 EBike[TN05E5] floor 1 slot 1 @ 2026-10-23T10:00
ENTER  Ticket#1006 DeliveryVan[TN06F6] floor 2 slot 0 @ 2026-10-23T08:30
ERROR: Duplicate registration number: TN02B2
ERROR: Unknown vehicle type: BUS
REPORT occupied=6/12 collection=Rs 0.0
EXIT   TN01A1 ticket#1001 parking=Rs 20.0 charging=Rs 0.0 total=Rs 20.0
EXIT   TN02B2 ticket#1002 parking=Rs 120.0 charging=Rs 0.0 total=Rs 120.0
EXIT   TN03C3 ticket#1003 parking=Rs 30.0 charging=Rs 120.0 total=Rs 150.0
EXIT   TN04D4 ticket#1004 parking=Rs 0.0 charging=Rs 0.0 total=Rs 0.0
EXIT   TN05E5 ticket#1005 parking=Rs 20.0 charging=Rs 0.0 total=Rs 20.0
```

```
EXIT   TN06F6 ticket#1006 parking=Rs 90.0 charging=Rs 0.0 total=Rs 90.0
ERROR: No active ticket for TN77Q7
REPORT occupied=0/12 collection=Rs 400.0
```

The run shows the three invalid cases being rejected without damaging the state: the duplicate `TN02B2`, the unknown type `BUS`, and an exit for a vehicle that never entered; the final report shows 0 of 12 slots occupied and a collection of Rs 400.

11.4 Defence: how well does the design follow the four pillars?

Principle	Evidence from my implementation
Encapsulation	All state is `private`; tickets are immutable; invalid objects cannot be built (tests T22-T31 prove invalid input is rejected); `regNo` normalisation happens in one constructor.
Inheritance	`Vehicle -> Car -> ElectricCar`: `ElectricCar` reuses `Car`'s rate and fee skeleton; adding `EBike` and `DeliveryVan` reused `calculateFee`, `equals`, `hashCode` and `toString` unchanged.
Polymorphism	`ParkingService.exit` has no `if (type == ...)`; the scenario priced 5 different categories through one call; dispatch trace in Task 8.
Abstraction	`Vehicle` (abstract) and `Chargeable`/`Pass` (interfaces) let the service depend on contracts, not on concrete classes; the registry (`registerType`) lets types be added without touching the service.

One limitation (stated honestly). The model assumes every slot suits every vehicle and does not represent charging bays, two-wheeler bays or slot reservations, and all data is held in memory (a restart loses all tickets). Fees are `double`, which is exact only while amounts stay whole rupees. The `DeliveryVan.calculateFee(int)` (clock-unaware overload) bills off-peak only; the service always uses the clock-aware overload, but a future caller could call the wrong one.

11.5 Alternative Concept and Complexity Analysis - active tickets in an ArrayList or a HashMap

Exit processing differs in how the ticket is found. In my service the tickets are in a `HashMap<String, ParkingTicket>` keyed by registration number; the assignment text assumes an array. The three solutions:

Three ways to find and remove the ticket on exit

```
// (1) array  ParkingTicket[] tickets;  int n;
for (int i = 0; i < n; i++)
    if (tickets[i].getVehicle().getRegNo().equals(reg)) { found = tickets[i]; tickets[i] = tickets[--n];
break; }

// (2) ArrayList<ParkingTicket>
for (int i = 0; i < list.size(); i++)
    if (list.get(i).getVehicle().getRegNo().equals(reg)) { found = list.remove(i); break; }   // remove =
O(n) shift

// (3) HashMap<String, ParkingTicket>   (used in ParkingService)
ParkingTicket found = map.remove(reg);                  // O(1) average; also gives the duplicate check for
free
```

Measured time to find a ticket (average per look-up, best of 5):

n active vehicles	Array scan	ArrayList scan	HashMap
1000	1.027 us		
10000	20.472 us		
100000	109.661 us		

Measured memory for n tickets (excluding the registration Strings):

n active vehicles	Array	ArrayList	HashMap
1000	20 KB		
10000	195 KB		
100000	1953 KB		

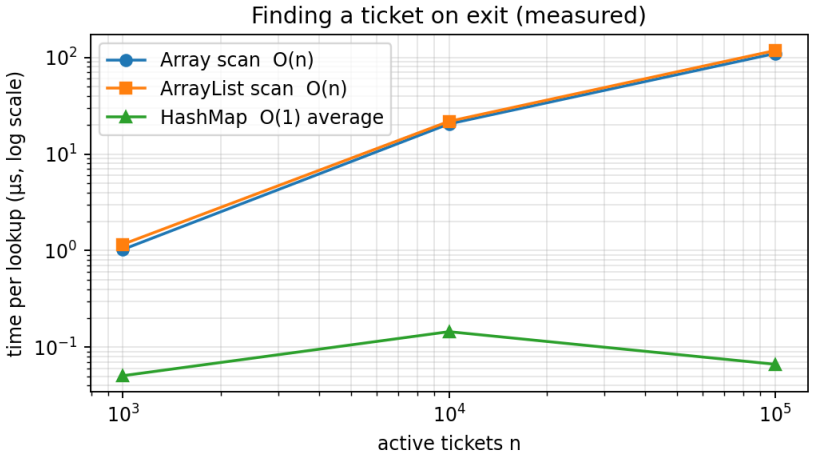


Figure 11.1 - Measured ticket look-up time

	Array	ArrayList	HashMap (mine)
Find on exit	`O(n)`	`O(n)`	`O(1)` average
Remove after exit	`O(1)` with swap-with-last, otherwise `O(n)` shift	`O(n)` (elements shift)	`O(1)` average
Duplicate-entry check	`O(n)` scan	`O(n)` scan	`O(1)` `containsKey`
Space for n tickets	`O(n)`, fixed capacity	`O(n)`, grows by about 1.5x	`O(n)` but about 2.7-2.9x larger (entry objects + table)

Conclusion. At n = 100,000 an array scan takes about 110 us per exit against 0.07 us for the HashMap (over 1,000x), and the gap widens linearly with n. For a few hundred vehicles the scan would still be unnoticeable, but the HashMap also answers the duplicate-registration rule in `O(1)`. The price is about 2.9x more memory (5,712 KB vs 1,953 KB at n = 100,000), which is acceptable. I therefore keep the HashMap.

TASK 12 - Re-evaluate and Adapt the Design (8 marks)

> **Decision in one line: MODIFY (a small, targeted change).** The original design is retained; **five new types are added** (`EBike`, `DeliveryVan`, `Pass`, `MonthlyPass`, `OneDayPass`) and **one existing class is modified** (`ParkingService.exit`, which took a `boolean hasMonthlyPass` and now takes a `Pass`). All other previously tested classes are unchanged.

A. Impact analysis (2 marks)

A.1 What has changed

Changed requirement	New demand on the design
E-bikes: flat fee	A fee that does **not** depend on hours: the pricing rule changes shape.
Delivery vans: double the car rate 09:00-17:00	A fee that depends on the **clock time**, not only on the duration: `calculateFee(int minutes)` cannot know the hour.
Visitors may buy a one-day pass	A **second kind** of pass (valid for one calendar day), not only the staff/student monthly pass.
3 to 5 floors, different number of slots on each	Floors are no longer uniform ({3,2,4,1,2}).

A.2 Which classes, methods and data structures are affected

Element	Affected?	Why
`Vehicle`, `TwoWheeler`, `Car`, `ElectricCar`	No	New categories are new subclasses; the overload `calculateFee(LocalTime, LocalTime)` already exists, so van pricing can override it.
`ParkingTicket`	No	It stores a `Vehicle` and times; it does not care about the category or the pass.
`ParkingFacility`	No	It already uses a jagged array; a new `int[]` {3,2,4,1,2} is just a new argument.
`ParkingService.enter`	No	Types are created through a registry: two `registerType` calls are added in start-up code.
`ParkingService.exit`	**Yes (modified)**	It checked a `boolean hasMonthlyPass`; two pass kinds need a `Pass` abstraction. One parameter and one condition change.
`Main` start-up	Yes (two lines)	Registers `EBIKE` and `VAN`.

A.3 Why different slots per floor affect the array design

A rectangular `new String[5][50]` or the formula `floor x slotsPerFloor + slot` assumes equal floor sizes. With {3,2,4,1,2} a rectangular array would create 25 slots for a 12-slot building (13 phantom slots that `allocate` would fill) and the 1-D formula would map `(floor 1, slot 2)` onto a slot of floor 2. The **jagged array** chosen in Task 2 already stores one array per floor with its own length, so no change was needed.

A.4 Which classes had to be modified rather than extended

Only **`ParkingService`**. It is the one class that hard-coded how a free-parking decision is made (`boolean`). Every other requirement was met by **extending** (new subclasses, a new interface implementation).

B. Evaluation of the original design (2 marks)

Question	Answer and evidence
Can new vehicle types be added only by writing new subclasses?	**Yes, and nothing else is needed.** `EBike` and `DeliveryVan` are two new files; the service creates them through the registry (`registerType("EBIKE", EBike::new)`).

Question	Answer and evidence
Are the pricing rules spread across many classes or in one place?	**Honestly: spread.** Each class holds its own rate (`ratePerHour`) and the cap/overlay rules are overridden in `EBike` and `DeliveryVan`; the common skeleton (ceiling, daily cap, past-midnight) is in one place, `Vehicle.calculateFee`. This is good for adding types, but weak if a rule must apply across types (e.g. a festival surcharge): it would then touch every class. That is the case for a `PricingRule` abstraction (see D, trade-offs).
Can the existing tested classes remain unchanged?	**Seven of eight can.** Unchanged: `Vehicle`, `TwoWheeler`, `Car`, `ElectricCar`, `ParkingTicket`, `ParkingFacility`, `Chargeable`. Modified: `ParkingService`. The earlier tests T01-T03 still give the same results.

C. Adaptation: revised class structure and implementation (2 marks)

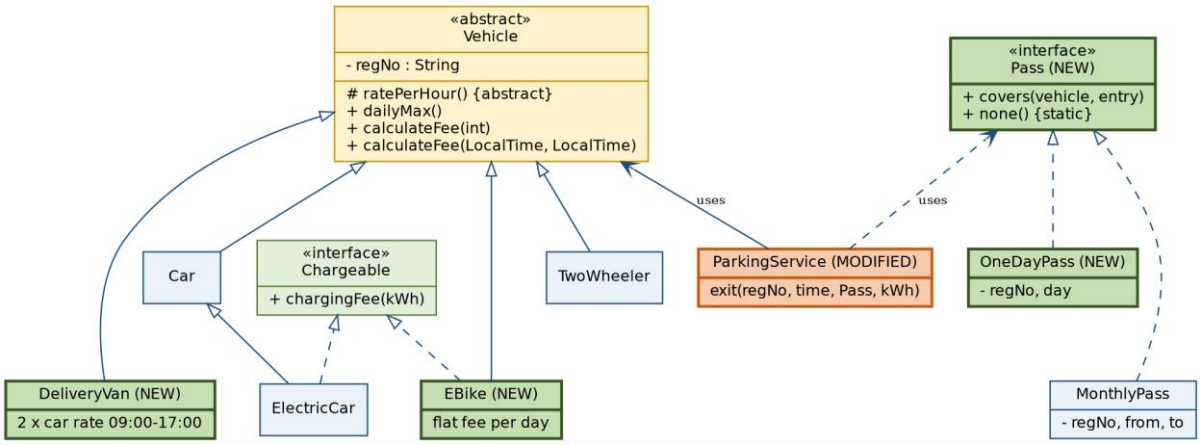


Figure 12.1 - Final class diagram: green = new, orange = modified, blue/yellow = unchanged

C.1 What has changed

- **New** `EBike` (flat Rs 20 per started day; also `Chargeable`).
- **New** `DeliveryVan` (Rs 30 off-peak, Rs 60 = double the car rate from 09:00 to 17:00 by the clock hour in which each billed hour starts; daily maximum 8 x 60 = 480).
- **New** `Pass` interface (`covers`, `describe` default method, `none()` static method); `MonthlyPass` now implements it; **new** `OneDayPass`.
- **Modified** `ParkingService.exit`: `exit(regNo, time, boolean hasMonthlyPass, kWh)` became `exit(regNo, time, Pass pass, kWh)`.
- **Facility** extended from 3 to 5 floors: only the argument changes: `new ParkingFacility(new int[]{3, 2, 4, 1, 2})`.

C.2 The new classes

parking/model/EBike.java (NEW)

```
package parking.model;
/** NEW (Task 12): flat fee per day, regardless of stay length. Can also charge. */
public final class EBike extends Vehicle implements Chargeable {
    private static final double FLAT_FEE = 20;
    public EBike(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return FLAT_FEE; }
    @Override public double dailyMax() { return FLAT_FEE; }
    @Override public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Negative duration: " + minutes);
        int days = minutes / MINUTES_PER_DAY;
        int rem = minutes % MINUTES_PER_DAY;
        return days * FLAT_FEE + (rem > 0 ? FLAT_FEE : 0);   // flat per started day
    }
    @Override public double chargingFee(int kWh) { return kWh * RATE_PER_KWH; }
}
```

parking/model/DeliveryVan.java (NEW)

```
package parking.model;
import java.time.LocalTime;
/** NEW (Task 12): double the car rate between 09:00 and 17:00 (by start of each billed hour). */
public final class DeliveryVan extends Vehicle {
    private static final int PEAK_START = 9, PEAK_END = 17;
    private static final double OFF_PEAK = 30, PEAK = 60;   // PEAK = 2 x Car rate
    public DeliveryVan(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return OFF_PEAK; }
    @Override public double dailyMax() { return PEAK * 8; }
    /** Clock-aware version: the hour-by-hour rate depends on when each hour starts. */
    @Override public double calculateFee(LocalTime in, LocalTime out) {
        int hours = (minutesBetween(in, out) + 59) / 60;
        double fee = 0;
        for (int h = 0; h < hours; h++) {
            int startHour = in.plusHours(h).getHour();
            fee += (startHour >= PEAK_START && startHour < PEAK_END) ? PEAK : OFF_PEAK;
        }
        return Math.min(fee, dailyMax());
    }
}
```

parking/model/Pass.java (NEW)

```
package parking.model;
import java.time.LocalDateTime;

/** NEW abstraction (Task 12): anything that can make parking free. */
public interface Pass {
    boolean covers(Vehicle v, LocalDateTime entry);
    default String describe() { return getClass().getSimpleName(); }
    static Pass none() { return (v, e) -> false; }       // no pass: nothing is free
}
```

parking/model/OneDayPass.java (NEW)

```
package parking.model;
import java.time.*;
/** NEW (Task 12): visitor one-day pass, valid for the calendar day of entry. */
public class OneDayPass implements Pass {
    private final String regNo; private final LocalDate day;
    public OneDayPass(String regNo, LocalDate day) { this.regNo = regNo.trim().toUpperCase(); this.day = day;
}
    @Override public boolean covers(Vehicle v, LocalDateTime entry) {
        return v.getRegNo().equals(regNo) && entry.toLocalDate().equals(day);
    }
}
```

C.3 The one modified method (ParkingService.exit)

BEFORE (original design)

```
public Receipt exit(String regNo, LocalDateTime when, boolean hasMonthlyPass, int kWh) {
    ...
    boolean free = hasMonthlyPass;
    ...
}
```

AFTER (Task 12): the only change in ParkingService

```
public Receipt exit(String regNo, LocalDateTime when, Pass pass, int kWh) {
    ...
    boolean free = pass != null && pass.covers(v, t.getEntryTime());
    ...
}
```

C.4 Start-up registration (the only other edit)

From parking.app.Main

```
s.registerType("EBIKE", EBike::new);        // Task 12: registered, service untouched
s.registerType("VAN",   DeliveryVan::new);  // Task 12: registered, service untouched
...
ParkingService svc = newService(3, 2, 4, 1, 2);   // five floors, different sizes
```

D. Evidence-based defence (2 marks)

Evidence	Result
Existing classes that had to change	**1 of 8** (`ParkingService`, one parameter and one condition). 7 unchanged: `Vehicle`, `TwoWheeler`, `Car`, `ElectricCar`, `ParkingTicket`, `ParkingFacility`, `Chargeable`.
New classes / interfaces written	5 (`EBike`, `DeliveryVan`, `Pass`, `MonthlyPass` as a `Pass`, `OneDayPass`), about 70 lines in total.
Test evidence for the new categories	T04 and T05: e-bike flat fee 20 for 20 min and for 7 h. T06, T07: van 90 and 120. T15, T16, T17: van peak boundaries at 17:00 and 09:00. Scenario: e-bike 20, van 90, one-day pass 0. **All 34 tests pass.**
Ease of adding a further type in future	One new subclass plus one `registerType` line; no edit to `Vehicle`, the service or the facility. Example below.

Adding "Bus" tomorrow: 4 lines + 1 registration

```
public final class Bus extends Vehicle {                    // hypothetical next category
    public Bus(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return 100; }
}
// start-up:  s.registerType("BUS", Bus::new);
```

Trade-offs and why the decision follows from the analysis

- **REDESIGN was rejected:** replacing the hierarchy with composition and a `PricingRule` strategy (Task 6.5) would have rewritten 7 working, tested classes to fix a problem the new requirements did not expose. **RETAIN was rejected:** pass handling really was hard-wired to one monthly-pass `boolean` and could not express a one-day pass without editing the service. **MODIFY** touches the minimum.
- **Cost of the choice 1:** pricing rules stay distributed (one rate per class). If a rule appears that cuts across vehicle types, I would extract a `PricingRule` interface and let `Vehicle` hold one: that would be the point at which a redesign pays.
- **Cost of the choice 2:** `DeliveryVan.calculateFee(int)` (the clock-unaware overload) bills at the off-peak rate only. The service always uses the clock-aware overload, and this is documented; a safer design would make the clock-unaware overload throw.
- **Cost of the choice 3:** `ParkingService` uses `instanceof Chargeable` for charging; acceptable because it asks about a **capability** (an interface), not about a concrete type.

APPENDIX A - Repository Layout and How to Run

Repository layout

```
campus-smart-parking/
|-- README.md
|-- input/sample_input.txt              input file for the Scanner runner
|-- task2/CastingDemo.java              Task 2  casting bug demo
|-- task3/ParkingFacility.java          Task 3  Complete the Code  (TODO 1, 2)
|-- task3/FeeSlabDemo.java              Task 3  switch / if-else slabs
|-- task5/StaticDemo.java               Task 5  static counter + GC demo
|-- task6/ConstructorTrace.java         Task 6  constructor order trace
|-- task8/FeeDemo.java                  Task 8  Complete the Code  (TODO 1, 2, 3)
|-- task10/Task10Demo.java              Task 10 validation + Integer cache
|-- src/parking/model/*.java            Vehicle ... ParkingTicket, Pass, ...
|-- src/parking/service/*.java          ParkingService, Receipt
|-- src/parking/app/*.java              Main (tests), Cli, Polymorphism
|-- bench/Benchmark.java                all complexity measurements
|-- outputs/*.txt                       captured console output of every program
|-- diagrams/*.dot, *.png               UML, flowcharts, memory, packages, charts
```

Commands (Java 17+ required; the code was run on Java 21)

```
# compile and run the standalone Complete-the-Code programs
javac -d out/task3 task3/*.java  &&  java -cp out/task3 ParkingFacility
javac -d out/task8 task8/*.java  &&  java -cp out/task8 FeeDemo

# compile and run the full system
javac -d out/system $(find src -name "*.java")
java -cp out/system parking.app.Main                    # scenario + 34 tests
java -cp out/system parking.app.Polymorphism            # overload / dispatch / equals
java -cp out/system parking.app.Cli "3,2,4,1,2" < input/sample_input.txt

# measurements
javac -d out/bench bench/Benchmark.java && java -Xmx2g -cp out/bench Benchmark
```

Git steps to publish the repository

Create an empty repository on GitHub first, then run

```
git init
git add .
git commit -m "Campus smart-parking: Tasks 1-12"
git branch -M main
git remote add origin https://github.com/<your-username>/campus-smart-parking.git
git push -u origin main
```

> **Tip for Task 12-D.** To show the "number of classes that had to change" as evidence, commit the version **before** the Task 12 changes (`git commit -m "v1 original design"`, `git tag v1`) and then the changes (`git tag v2`); `git diff v1 v2 --stat` then shows exactly which files changed. The final code in this report is the **v2** state.

APPENDIX B - Supporting Source Listings

parking/app/Main.java (scenario + test harness)

```
package parking.app;

import parking.model.*;
import parking.service.*;
import java.time.*;
import java.util.*;
import java.util.function.Supplier;

public class Main {
    static final LocalDate D = LocalDate.of(2026, 10, 23);
    static int pass = 0, fail = 0, no = 0;

    static LocalDateTime at(int h, int m) { return LocalDateTime.of(D, LocalTime.of(h, m)); }

    static ParkingService newService(int... floors) {
        ParkingService s = new ParkingService(new ParkingFacility(floors));
        s.registerType("TW", TwoWheeler::new);
        s.registerType("CAR", Car::new);
        s.registerType("EV", ElectricCar::new);
        s.registerType("EBIKE", EBike::new);        // Task 12: registered, service untouched
        s.registerType("VAN", DeliveryVan::new);    // Task 12: registered, service untouched
        return s;
    }

    static void row(String cat, String desc, Object exp, Object act) {
        boolean ok = String.valueOf(exp).equals(String.valueOf(act));
        if (ok) pass++; else fail++;
        System.out.printf("T%02d | %-9s | %-52s | %-26s | %-26s | %s%n",
                ++no, cat, desc, exp, act, ok ? "PASS" : "FAIL");
    }
    static void check(String cat, String desc, Object exp, Supplier<Object> act) {
        Object a; try { a = act.get(); } catch (RuntimeException e) { a = "EXC " +
e.getClass().getSimpleName(); }
        row(cat, desc, exp, a);
    }
    static void thrown(String cat, String desc, Class<? extends RuntimeException> exp, Runnable r) {
        String a;
        try { r.run(); a = "no exception"; } catch (RuntimeException e) { a = e.getClass().getSimpleName(); }
        row(cat, desc, exp.getSimpleName(), a);
    }

    public static void main(String[] args) {
        // ---------------- hand-calculated scenario ----------------
        System.out.println("=== SCENARIO: facility 5 floors {3,2,4,1,2} = 12 slots, 23-Oct-2026 ===");
        ParkingService svc = newService(3, 2, 4, 1, 2);
        String[][] in = {
            {"TW","TN01A1"},{"CAR","TN02B2"},{"EV","TN03C3"},{"CAR","TN04D4"},
            {"EBIKE","TN05E5"},{"VAN","TN06F6"},{"CAR","TN07G7"} };
        int[][] t_in  = {{8,0},{8,15},{9,0},{8,0},{10,0},{8,30},{11,0}};
        int[][] t_out = {{9,30},{11,16},{10,0},{18,0},{10,20},{10,30},{13,0}};
        int[] kwh = {0,0,10,0,0,0,0};
        Pass[] passes = { Pass.none(), Pass.none(), Pass.none(),
            new MonthlyPass("TN04D4", D.minusDays(10), D.plusDays(20)),
            Pass.none(), Pass.none(), new OneDayPass("TN07G7", D) };
        double[] hand = {20, 120, 150, 0, 20, 90, 0};
        for (int i = 0; i < in.length; i++) {
            ParkingTicket t = svc.enter(in[i][0], in[i][1], at(t_in[i][0], t_in[i][1]));
            System.out.println("ENTER  " + t);
        }
        System.out.println("Occupied now: " + svc.facility().occupied() + " / " + svc.facility().capacity());
        double handTotal = 0;
        for (int i = 0; i < in.length; i++) {
            Receipt r = svc.exit(in[i][1], at(t_out[i][0], t_out[i][1]), passes[i], kwh[i]);
            handTotal += hand[i];
            System.out.printf("EXIT   %s ticket#%d parking=%.1f charging=%.1f total=%.1f (hand=%.1f)%n",
                    in[i][1], r.ticketNo(), r.parkingFee(), r.chargingFee(), r.total(), hand[i]);
        }
        System.out.printf("Hand-calculated collection = %.1f | Program collection = %.1f | match = %b%n",
                handTotal, svc.totalCollection(), handTotal == svc.totalCollection());

        // ---------------- test table ----------------
        System.out.println();
        System.out.println("=== TEST TABLE ===");
        System.out.println("ID  | Category  | Description                                                    | Expected
| Actual                    | Result");
        Car car = new Car("X"); EBike eb = new EBike("E"); DeliveryVan van = new DeliveryVan("V");
        // normal
```

```
        check("Normal", "TwoWheeler, 130 min", 30.0, () -> new TwoWheeler("A").calculateFee(130));
        check("Normal", "Car, 130 min", 90.0, () -> car.calculateFee(130));
        check("Normal", "ElectricCar 130 min + 10 kWh", 210.0,
              () -> new ElectricCar("B").calculateFee(130) + new ElectricCar("B").chargingFee(10));
        check("Normal", "EBike, 20 min -> flat fee", 20.0, () -> eb.calculateFee(20));
        check("Normal", "EBike, 7 h -> same flat fee", 20.0, () -> eb.calculateFee(420));
        check("Normal", "Van 08:30-10:30 (1 off-peak h + 1 peak h)", 90.0,
              () -> van.calculateFee(LocalTime.of(8, 30), LocalTime.of(10, 30)));
        check("Normal", "Van 10:00-12:00 (2 peak hours)", 120.0,
              () -> van.calculateFee(LocalTime.of(10, 0), LocalTime.of(12, 0)));
        check("Normal", "Monthly pass covers valid regNo/date", true,
              () -> new MonthlyPass("tn04d4", D.minusDays(1), D.plusDays(1)).covers(new Car("TN04D4"), at(9,
0)));
        check("Normal", "Monthly pass expired yesterday", false,
              () -> new MonthlyPass("TN04D4", D.minusDays(9), D.minusDays(1)).covers(new Car("TN04D4"), at(9,
0)));
        // boundary
        check("Boundary", "Zero-minute stay (Car)", 0.0, () -> car.calculateFee(0));
        check("Boundary", "Exactly 60 min (Car) = 1 hour", 30.0, () -> car.calculateFee(60));
        check("Boundary", "61 min (Car) = 2 hours", 60.0, () -> car.calculateFee(61));
        check("Boundary", "Exactly 8 h (Car) = daily max", 240.0, () -> car.calculateFee(480));
        check("Boundary", "9 h (Car) capped at daily max", 240.0, () -> car.calculateFee(540));
        check("Boundary", "Van 16:30-17:30 = 1 h starting in peak", 60.0,
              () -> van.calculateFee(LocalTime.of(16, 30), LocalTime.of(17, 30)));
        check("Boundary", "Van 16:30-17:31 (2nd hour starts 17:30)", 90.0,
              () -> van.calculateFee(LocalTime.of(16, 30), LocalTime.of(17, 31)));
        check("Boundary", "Van 08:00-09:00 (peak starts at 09:00)", 30.0,
              () -> van.calculateFee(LocalTime.of(8, 0), LocalTime.of(9, 0)));
        check("Boundary", "Car 23:00 -> 01:00 past midnight", 60.0,
              () -> car.calculateFee(LocalTime.of(23, 0), LocalTime.of(1, 0)));
        check("Boundary", "Car exactly 24 h = one daily max", 240.0, () -> {
            ParkingService s = newService(1);
            s.enter("CAR", "Z1", at(10, 0));
            return s.exit("Z1", at(10, 0).plusDays(1), Pass.none(), 0).total(); });
        check("Boundary", "Last free slot used, next entry rejected", "EXC IllegalStateException", () -> {
            ParkingService s = newService(1);
            s.enter("CAR", "L1", at(9, 0));
            return s.enter("CAR", "L2", at(9, 5)); });
        check("Boundary", "Slot is reused after the vehicle leaves", "[0, 0]", () -> {
            ParkingService s = newService(1);
            s.enter("CAR", "R1", at(9, 0)); s.exit("R1", at(9, 30), Pass.none(), 0);
            ParkingTicket t = s.enter("CAR", "R2", at(10, 0));
            return Arrays.toString(new int[]{t.getFloor(), t.getSlot()}); });
        // invalid
        ParkingService inv = newService(2, 2);
        inv.enter("CAR", "D1", at(9, 0));
        thrown("Invalid", "Unknown vehicle type BUS", IllegalArgumentException.class, () -> inv.enter("BUS",
"B1", at(9, 0)));
        thrown("Invalid", "Duplicate registration number", IllegalStateException.class, () -> inv.enter("TW",
"d1", at(9, 5)));
        thrown("Invalid", "Blank registration number", IllegalArgumentException.class, () -> inv.enter("CAR",
"  ", at(9, 5)));
        thrown("Invalid", "Negative duration", IllegalArgumentException.class, () -> car.calculateFee(-5));
        thrown("Invalid", "Exit of a vehicle that never entered", IllegalStateException.class, () ->
inv.exit("NOPE", at(10, 0), Pass.none(), 0));
        thrown("Invalid", "Exit time before entry time", IllegalArgumentException.class, () -> inv.exit("D1",
at(8, 0), Pass.none(), 0));
        thrown("Invalid", "Charging requested for a Car", IllegalArgumentException.class, () ->
inv.exit("D1", at(10, 0), Pass.none(), 5));
        ParkingFacility pf = new ParkingFacility(new int[]{2, 1});
        pf.allocate("Q1");
        check("Invalid", "release(0,0) valid then repeated", "true false", () -> pf.release(0, 0) + " " +
pf.release(0, 0));
        check("Invalid", "release(5,0) / release(0,-1) / release(1,9)", "false false false",
              () -> pf.release(5, 0) + " " + pf.release(0, -1) + " " + pf.release(1, 9));
        thrown("Invalid", "Facility with a zero-slot floor", IllegalArgumentException.class, () -> new
ParkingFacility(new int[]{3, 0}));
        // object contract
        check("Contract", "Car(TN02B2).equals(ElectricCar(tn02b2))", true, () -> new Car("TN02B2").equals(new
ElectricCar("tn02b2")));
        check("Contract", "Ticket numbers strictly increasing/unique", true, () -> {
            ParkingTicket a = new ParkingTicket(car, 0, 0, at(9, 0)), b = new ParkingTicket(car, 0, 1, at(9,
0));
            return b.getTicketNo() == a.getTicketNo() + 1; });
        check("Contract", "Copy constructor keeps number, flags duplicate", "true true", () -> {
            ParkingTicket a = new ParkingTicket(car, 0, 0, at(9, 0)), c = new ParkingTicket(a);
            return (c.getTicketNo() == a.getTicketNo()) + " " + c.isDuplicate(); });
        System.out.println();
        System.out.printf("SUMMARY: %d tests, %d passed, %d failed%n", pass + fail, pass, fail);
    }
}
```

parking/app/Cli.java (Scanner-driven runner)

```
package parking.app;

import parking.model.*;
import parking.service.*;
import java.time.*;
import java.util.*;

/**
 * Input-driven runner (Scanner on System.in).  Floors come from args[0], e.g. "3,2,4,1,2".
 *   ENTER <TYPE> <REG> <HH:mm>
 *   EXIT  <REG> <HH:mm> [kWh] [MONTHLY|ONEDAY]
 *   REPORT
 */
public class Cli {
    static final LocalDate DAY = LocalDate.of(2026, 10, 23);

    public static void main(String[] args) {
        String[] parts = (args.length > 0 ? args[0] : "3,2,4,1,2").split(",");
        int[] floors = new int[parts.length];
        for (int i = 0; i < parts.length; i++) floors[i] = Integer.parseInt(parts[i].trim());
        ParkingService svc = Main.newService(floors);
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty() || line.startsWith("#")) continue;
            String[] t = line.split("\\s+");
            try {
                switch (t[0].toUpperCase()) {
                    case "ENTER" -> System.out.println("ENTER  " + svc.enter(t[1], t[2],
DAY.atTime(LocalTime.parse(t[3]))));
                    case "EXIT" -> {
                        int kwh = t.length > 3 ? Integer.parseInt(t[3]) : 0;
                        String kind = t.length > 4 ? t[4].toUpperCase() : "NONE";
                        Pass p = switch (kind) {
                            case "MONTHLY" -> new MonthlyPass(t[1], DAY.minusDays(5), DAY.plusDays(25));
                            case "ONEDAY" -> new OneDayPass(t[1], DAY);
                            default -> Pass.none();
                        };
                        Receipt r = svc.exit(t[1], DAY.atTime(LocalTime.parse(t[2])), p, kwh);
                        System.out.printf("EXIT   %s ticket#%d parking=Rs %.1f charging=Rs %.1f total=Rs
%.1f%n",
                                t[1], r.ticketNo(), r.parkingFee(), r.chargingFee(), r.total());
                    }
                    case "REPORT" -> System.out.printf("REPORT occupied=%d/%d collection=Rs %.1f%n",
                            svc.facility().occupied(), svc.facility().capacity(), svc.totalCollection());
                    default -> System.out.println("ERROR: unknown command " + t[0]);
                }
            } catch (RuntimeException e) {
                System.out.println("ERROR: " + e.getMessage());
            }
        }
    }
}
```

parking/app/Polymorphism.java

```
package parking.app;

import parking.model.*;
import java.time.LocalTime;
import java.util.*;

/** Tasks 6-9 demonstration: overloading, upcasting, dynamic dispatch, equals/hashCode. */
public class Polymorphism {
    public static void main(String[] args) {
        Car car = new Car("TN02B2");
        System.out.println("--- Compile-time polymorphism (overloading) ---");
        System.out.println("calculateFee(130)                          = " + car.calculateFee(130));
        System.out.println("calculateFee(LocalTime 08:00, 10:10)      = "
                + car.calculateFee(LocalTime.of(8, 0), LocalTime.of(10, 10)));
        System.out.println("calculateFee(23:00 -> 01:00, past midnight) = "
                + car.calculateFee(LocalTime.of(23, 0), LocalTime.of(1, 0)));

        System.out.println("--- Upcasting: one Vehicle[] holds every category ---");
        Vehicle[] lot = { new TwoWheeler("TN01A1"), new Car("TN02B2"), new ElectricCar("TN03C3"),
                          new EBike("TN05E5"), new DeliveryVan("TN06F6") };

        System.out.println("--- Runtime polymorphism (dynamic method dispatch) ---");
        System.out.printf("%-9s %-13s %-27s %-26s %s%n", "Declared", "Actual", "calculateFee(int) runs in",
```

```
                "ratePerHour() runs in", "fee(130 min)");
        for (Vehicle v : lot) {
            System.out.printf("%-9s %-13s %-27s %-26s %.1f%n", "Vehicle", v.getClass().getSimpleName(),
                    declaring(v, "calculateFee", int.class), declaring(v, "ratePerHour"),
v.calculateFee(130));
        }

        System.out.println("--- equals / hashCode ---");
        Vehicle a = new Car("tn02b2"), b = new ElectricCar("TN02B2 ");
        System.out.println("Car(\"tn02b2\").equals(ElectricCar(\"TN02B2 \")) = " + a.equals(b));
        System.out.println("hash codes equal                                 = " + (a.hashCode() ==
b.hashCode()));
        Set<Vehicle> set = new HashSet<>(List.of(a));
        System.out.println("HashSet already contains the 'other' object    = " + set.contains(b));
        System.out.println("toString                                         = " + a + " / " + b);
    }

    /** Walks up the class chain to find which class really declares the method that runs. */
    static String declaring(Object o, String name, Class<?>... types) {
        for (Class<?> c = o.getClass(); c != null; c = c.getSuperclass()) {
            try { c.getDeclaredMethod(name, types); return c.getSimpleName(); }
            catch (NoSuchMethodException ignored) { }
        }
        return "?";
    }
}
```

APPENDIX C - Benchmark Program and Raw Measurements

Run environment: 21.0.10 / 1 cpu. Each timing is the best of 5 timed rounds after warm-up, to reduce JIT and GC noise; memory figures are the JVM heap before and after allocation. Absolute values depend on the machine; the growth trends are the evidence.

Raw output of bench/Benchmark.java

```
ENV|java|21.0.10 / 1 cpu
B1|find regNo n=1000 (best of 5)|objects 2.33 us | parallel arrays 2.32 us
B1|find regNo n=10000 (best of 5)|objects 22.23 us | parallel arrays 20.26 us
B1|find regNo n=100000 (best of 5)|objects 270.63 us | parallel arrays 263.89 us
B1|extra space per vehicle (excl. the regNo String itself, n=200000)|parallel arrays 8.0 B | objects 28.0 B
B2|locate slot (ns/op, 100 floors of 20-99 slots)|jagged 1.42 | 1-D with offsets 1.36 | ArrayList<String[]>
3.52
B2|global index -> floor (binary search O(log F)), ns/op|38.08
B2|space for 100 floors, 6056 slots (compressed oops, 4-byte refs)|jagged: 26240 B | 1-D: 24660 B |
ArrayList<String[]>: 26280 B
B3|fill all 20 slots (1x20) avg ns per allocate()|scan 15.1 | counters+stack 20.2
B3|fill all 150 slots (3x50) avg ns per allocate()|scan 30.5 | counters+stack 12.7
B3|fill all 500 slots (5x100) avg ns per allocate()|scan 97.0 | counters+stack 8.5
B3|fill all 10000 slots (20x500) avg ns per allocate()|scan 3170.2 | counters+stack 7.2
B3|fill all 50000 slots (50x1000) avg ns per allocate()|scan 8033.2 | counters+stack 15.5
B4|create n=100000 tickets (ms, best of 5)|ctor chain 0.5 | default+setters 0.4 | static factory 0.4
B4|create n=1000000 tickets (ms, best of 5)|ctor chain 18.1 | default+setters 18.2 | static factory 17.6
B4|create n=5000000 tickets (ms, best of 5)|ctor chain 115.6 | default+setters 106.5 | static factory 110.0
B8|fee per call, k=3 types (ns)|dynamic dispatch 14.52 | switch on type 9.33 | instanceof chain 12.35
B8|fee per call, k=8 types (ns)|dynamic dispatch 18.10 | switch on type 12.45 | instanceof chain 14.09
B10|update a field (ns/op, best of 5)|validating setter 0.550 | public field, no check 0.482
B11|find ticket on exit n=1000 (avg us per lookup, best of 5)|array scan 1.027 | ArrayList scan 1.162 |
HashMap 0.0507
B11|memory for n=1000 tickets (KB, excl. regNo Strings)|array 20 | ArrayList 20 | HashMap 55
B11|find ticket on exit n=10000 (avg us per lookup, best of 5)|array scan 20.472 | ArrayList scan 21.769 |
HashMap 0.1448
B11|memory for n=10000 tickets (KB, excl. regNo Strings)|array 195 | ArrayList 211 | HashMap 533
B11|find ticket on exit n=100000 (avg us per lookup, best of 5)|array scan 109.661 | ArrayList scan 117.749 |
HashMap 0.0666
B11|memory for n=100000 tickets (KB, excl. regNo Strings)|array 1953 | ArrayList 1979 | HashMap 5712
```

bench/Benchmark.java

```
import java.util.*;

/** Measurements for the "Alternative Concept and Complexity Analysis" parts. */
public class Benchmark {
    static volatile long sink;
    static long usedMem() { for (int i = 0; i < 4; i++) System.gc(); Runtime r = Runtime.getRuntime(); return
r.totalMemory() - r.freeMemory(); }
    static void out(String k, String label, String v) { System.out.println(k + "|" + label + "|" + v); }

    // ---------------- B1 : object vs parallel arrays ----------------
    static class Veh { String regNo; String type; int entry; Veh(String r, String t, int e) { regNo = r; type
= t; entry = e; } }
    static void b1() {
        for (int n : new int[]{1_000, 10_000, 100_000}) {
            Veh[] objs = new Veh[n]; String[] reg = new String[n], type = new String[n]; int[] entry = new
int[n];
            for (int i = 0; i < n; i++) { String r = "TN" + i; objs[i] = new Veh(r, "CAR", i); reg[i] = r;
type[i] = "CAR"; entry[i] = i; }
            String target = "TN" + (n - 1);             // worst case: last element
            int reps = 20_000_000 / n; long t0, t1, t2;
            for (int w = 0; w < 3; w++) { find(objs, target); find(reg, target); }
            long bo = Long.MAX_VALUE, bp = Long.MAX_VALUE;
            for (int round = 0; round < 5; round++) {
                t0 = System.nanoTime(); for (int i = 0; i < reps; i++) sink += find(objs, target);
                t1 = System.nanoTime(); for (int i = 0; i < reps; i++) sink += find(reg, target);
                t2 = System.nanoTime(); bo = Math.min(bo, t1 - t0); bp = Math.min(bp, t2 - t1);
            }
            out("B1", "find regNo n=" + n + " (best of 5)", String.format("objects %.2f us | parallel arrays
%.2f us", bo / 1e3 / reps, bp / 1e3 / reps));
        }
        int n = 200_000; long m0 = usedMem();
        String[] reg = new String[n]; for (int i = 0; i < n; i++) reg[i] = "TN" + i;
        long m1 = usedMem(); String[] ty = new String[n]; int[] en = new int[n]; Arrays.fill(ty, "CAR");
        long m2 = usedMem(); Veh[] ob = new Veh[n]; for (int i = 0; i < n; i++) ob[i] = new Veh(reg[i],
"CAR", i);
        long m3 = usedMem();
        out("B1", "extra space per vehicle (excl. the regNo String itself, n=200000)",
```

```
            String.format("parallel arrays %.1f B | objects %.1f B", (m2 - m1) / (double) n, (m3 - m2) /
(double) n));
        sink += reg.length + ty.length + en.length + ob.length;
    }
    static int find(Veh[] a, String k) { for (int i = 0; i < a.length; i++) if (a[i].regNo.equals(k)) return
i; return -1; }
    static int find(String[] a, String k) { for (int i = 0; i < a.length; i++) if (a[i].equals(k)) return i;
return -1; }

    // ---------------- B2 : jagged vs 1-D vs ArrayList ----------------
    static void b2() {
        int F = 100; int[] size = new int[F]; Random rnd = new Random(1);
        for (int f = 0; f < F; f++) size[f] = 20 + rnd.nextInt(80);
        String[][] jag = new String[F][]; int[] off = new int[F + 1];
        List<String[]> list = new ArrayList<>();
        for (int f = 0; f < F; f++) { jag[f] = new String[size[f]]; off[f + 1] = off[f] + size[f];
list.add(jag[f]); }
        String[] flat = new String[off[F]];
        int N = 5_000_000; int[] ff = new int[N], ss = new int[N];
        for (int i = 0; i < N; i++) { ff[i] = rnd.nextInt(F); ss[i] = rnd.nextInt(size[ff[i]]); }
        for (int w = 0; w < 3; w++) { long x = 0; for (int i = 0; i < N; i++) { x += (jag[ff[i]][ss[i]] ==
null ? 1 : 0) + (flat[off[ff[i]] + ss[i]] == null ? 1 : 0) + (list.get(ff[i])[ss[i]] == null ? 1 : 0); sink
+= x; } }
        long t0 = System.nanoTime(); long x = 0;
        for (int i = 0; i < N; i++) x += jag[ff[i]][ss[i]] == null ? 1 : 0;
        long t1 = System.nanoTime();
        for (int i = 0; i < N; i++) x += flat[off[ff[i]] + ss[i]] == null ? 1 : 0;
        long t2 = System.nanoTime();
        for (int i = 0; i < N; i++) x += list.get(ff[i])[ss[i]] == null ? 1 : 0;
        long t3 = System.nanoTime(); sink += x;
        out("B2", "locate slot (ns/op, 100 floors of 20-99 slots)", String.format("jagged %.2f | 1-D with
offsets %.2f | ArrayList<String[]> %.2f", (t1 - t0) / (double) N, (t2 - t1) / (double) N, (t3 - t2) /
(double) N));
        // 1-D: global index -> (floor, slot) needs a search of the offsets
        int[] g = new int[N]; for (int i = 0; i < N; i++) g[i] = rnd.nextInt(off[F]);
        t0 = System.nanoTime(); long y = 0;
        for (int i = 0; i < N; i++) { int p = Arrays.binarySearch(off, g[i]); int fl = p >= 0 ? p : -p - 2; y
+= fl; }
        t1 = System.nanoTime(); sink += y;
        out("B2", "global index -> floor (binary search O(log F)), ns/op", String.format("%.2f", (t1 - t0) /
(double) N));
        int total = off[F];
        out("B2", "space for 100 floors, " + total + " slots (compressed oops, 4-byte refs)",
            "jagged: " + (total * 4 + F * 16 + 16 + F * 4) + " B | 1-D: " + (total * 4 + 16 + (F + 1) * 4 +
16) + " B | ArrayList<String[]>: " + (total * 4 + F * 16 + 16 + 16 + F * 4 + 24) + " B");
    }

    // ---------------- B3 : allocate scan vs counters + stack ----------------
    static class Stacked {
        final int[] free; final int[][] stack; final int[] top; final String[][] slots;
        Stacked(int F, int S) { free = new int[F]; stack = new int[F][S]; top = new int[F]; slots = new
String[F][S];
            for (int f = 0; f < F; f++) { free[f] = S; top[f] = S; for (int s = 0; s < S; s++) stack[f][s] =
S - 1 - s; } }
        boolean allocate(String v) { for (int f = 0; f < free.length; f++) if (free[f] > 0) { int s =
stack[f][--top[f]]; slots[f][s] = v; free[f]--; return true; } return false; }
        void release(int f, int s) { slots[f][s] = null; stack[f][top[f]++] = s; free[f]++; }
    }
    static void b3() {
        int[][] cfg = {{1, 20}, {3, 50}, {5, 100}, {20, 500}, {50, 1000}};
        for (int[] c : cfg) {
            int F = c[0], S = c[1], N = F * S;
            int reps = Math.max(1, 3_000_000 / N);
            if (N >= 10_000) reps = 1;
            // warm up
            for (int w = 0; w < 3; w++) { scanFill(F, S); stackFill(F, S); }
            long t0 = System.nanoTime(); for (int r = 0; r < reps; r++) scanFill(F, S);
            long t1 = System.nanoTime(); for (int r = 0; r < reps; r++) stackFill(F, S);
            long t2 = System.nanoTime();
            out("B3", "fill all " + N + " slots (" + F + "x" + S + ") avg ns per allocate()",
String.format("scan %.1f | counters+stack %.1f", (t1 - t0) / 1.0 / reps / N, (t2 - t1) / 1.0 / reps / N));
        }
    }
    static void scanFill(int F, int S) { String[][] sl = new String[F][S]; String v = "V"; for (int i = 0; i
< F * S; i++) { out: for (int f = 0; f < F; f++) for (int s = 0; s < S; s++) if (sl[f][s] == null) { sl[f][s]
= v; break out; } } sink += sl.length; }
    static void stackFill(int F, int S) { Stacked st = new Stacked(F, S); for (int i = 0; i < F * S; i++)
st.allocate("V"); sink += st.free[0]; }

    // ---------------- B4 : constructors vs setters vs factory ----------------
    static class TA { final int no, floor, slot; final String reg; TA(String r, int f, int s) { this(r, f, s,
0); } TA(String r, int f, int s, int n) { if (r == null || f < 0 || s < 0) throw new
IllegalArgumentException(); reg = r; floor = f; slot = s; no = n; } }
```

```
    static class TB { int no, floor, slot; String reg; TB() {} void setReg(String r) { if (r == null) throw
new IllegalArgumentException(); reg = r; } void setFloor(int f) { if (f < 0) throw new
IllegalArgumentException(); floor = f; } void setSlot(int s) { if (s < 0) throw new
IllegalArgumentException(); slot = s; } }
    static class TC { final int no, floor, slot; final String reg; private TC(String r, int f, int s) { reg =
r; floor = f; slot = s; no = 0; } static TC of(String r, int f, int s) { if (r == null || f < 0 || s < 0)
throw new IllegalArgumentException(); return new TC(r, f, s); } }
    static void b4() {
        for (int n : new int[]{100_000, 1_000_000, 5_000_000}) {
            Object[] hold = new Object[n]; long bA = Long.MAX_VALUE, bB = Long.MAX_VALUE, bC =
Long.MAX_VALUE;
            for (int round = 0; round < 6; round++) {
                System.gc();
                long a = System.nanoTime(); for (int i = 0; i < n; i++) hold[i] = new TA("R", i & 7, i & 15);
                long b = System.nanoTime(); for (int i = 0; i < n; i++) hold[i] = null; System.gc();
                long c = System.nanoTime(); for (int i = 0; i < n; i++) { TB x = new TB(); x.setReg("R");
x.setFloor(i & 7); x.setSlot(i & 15); hold[i] = x; }
                long d = System.nanoTime(); for (int i = 0; i < n; i++) hold[i] = null; System.gc();
                long e = System.nanoTime(); for (int i = 0; i < n; i++) hold[i] = TC.of("R", i & 7, i & 15);
                long f = System.nanoTime(); for (int i = 0; i < n; i++) hold[i] = null;
                if (round > 0) { bA = Math.min(bA, b - a); bB = Math.min(bB, d - c); bC = Math.min(bC, f -
e); }
            }
            sink += hold.length;
            out("B4", "create n=" + n + " tickets (ms, best of 5)", String.format("ctor chain %.1f |
default+setters %.1f | static factory %.1f", bA / 1e6, bB / 1e6, bC / 1e6));
        }
    }

    // ---------------- B6-B8 : dispatch vs switch vs instanceof ----------------
    static abstract class V { abstract double rate(); double fee(int m) { return ((m + 59) / 60) * rate(); }
}
    static class V0 extends V { double rate() { return 10; } } static class V1 extends V { double rate() {
return 30; } }
    static class V2 extends V { double rate() { return 35; } } static class V3 extends V { double rate() {
return 20; } }
    static class V4 extends V { double rate() { return 60; } } static class V5 extends V { double rate() {
return 45; } }
    static class V6 extends V { double rate() { return 25; } } static class V7 extends V { double rate() {
return 15; } }
    static double feeSwitch(int type, int m) { int h = (m + 59) / 60; switch (type) { case 0: return h * 10;
case 1: return h * 30; case 2: return h * 35; case 3: return h * 20; case 4: return h * 60; case 5: return h
* 45; case 6: return h * 25; default: return h * 15; } }
    static double feeInstanceof(V v, int m) { int h = (m + 59) / 60; if (v instanceof V0) return h * 10; else
if (v instanceof V1) return h * 30; else if (v instanceof V2) return h * 35; else if (v instanceof V3) return
h * 20; else if (v instanceof V4) return h * 60; else if (v instanceof V5) return h * 45; else if (v
instanceof V6) return h * 25; else return h * 15; }
    static void b8() {
        int N = 4_000_000; Random r = new Random(7);
        for (int k : new int[]{3, 8}) {
            V[] vs = new V[N]; int[] ty = new int[N]; int[] mins = new int[N];
            for (int i = 0; i < N; i++) { int t = r.nextInt(k); ty[i] = t; mins[i] = r.nextInt(600);
                vs[i] = switch (t) { case 0 -> new V0(); case 1 -> new V1(); case 2 -> new V2(); case 3 ->
new V3(); case 4 -> new V4(); case 5 -> new V5(); case 6 -> new V6(); default -> new V7(); }; }
            for (int w = 0; w < 3; w++) { double x = 0; for (int i = 0; i < N; i++) x += vs[i].fee(mins[i]) +
feeSwitch(ty[i], mins[i]) + feeInstanceof(vs[i], mins[i]); sink += (long) x; }
            double x = 0; long a = System.nanoTime(); for (int i = 0; i < N; i++) x += vs[i].fee(mins[i]);
            long b = System.nanoTime(); for (int i = 0; i < N; i++) x += feeSwitch(ty[i], mins[i]);
            long c = System.nanoTime(); for (int i = 0; i < N; i++) x += feeInstanceof(vs[i], mins[i]);
            long d = System.nanoTime(); sink += (long) x;
            out("B8", "fee per call, k=" + k + " types (ns)", String.format("dynamic dispatch %.2f | switch
on type %.2f | instanceof chain %.2f", (b - a) / (double) N, (c - b) / (double) N, (d - c) / (double) N));
        }
    }

    // ---------------- B10 : validation cost ----------------
    static class Pv { private final int[] d = new int[1024]; void set(int i, int v) { if (v < 0) throw new
IllegalArgumentException(); d[i] = v; } int sum() { int t = 0; for (int x : d) t += x; return t; } }
    static class Pp { public final int[] d = new int[1024]; int sum() { int t = 0; for (int x : d) t += x;
return t; } }
    static void b10() {
        int N = 200_000_000; Pv a = new Pv(); Pp b = new Pp(); long bA = Long.MAX_VALUE, bB = Long.MAX_VALUE;
        for (int round = 0; round < 6; round++) {
            long t0 = System.nanoTime(); for (int i = 0; i < N; i++) a.set(i & 1023, i & 4095);
            long t1 = System.nanoTime(); for (int i = 0; i < N; i++) b.d[i & 1023] = i & 4095;
            long t2 = System.nanoTime();
            if (round > 0) { bA = Math.min(bA, t1 - t0); bB = Math.min(bB, t2 - t1); }
        }
        sink += a.sum() + b.sum();
        out("B10", "update a field (ns/op, best of 5)", String.format("validating setter %.3f | public field,
no check %.3f", bA / (double) N, bB / (double) N));
    }

    // ---------------- B11 : array vs ArrayList vs HashMap on exit ----------------
```

```
    static class Tk { final String reg; Tk(String r) { reg = r; } }
    static void b11() {
        for (int n : new int[]{1_000, 10_000, 100_000}) {
            String[] regs = new String[n]; for (int i = 0; i < n; i++) regs[i] = "TN" + i;
            long m0 = usedMem(); Tk[] arr = new Tk[n]; for (int i = 0; i < n; i++) arr[i] = new Tk(regs[i]);
            long m1 = usedMem(); List<Tk> list = new ArrayList<>(); for (int i = 0; i < n; i++) list.add(new
Tk(regs[i]));
            long m2 = usedMem(); Map<String, Tk> map = new HashMap<>(); for (int i = 0; i < n; i++)
map.put(regs[i], new Tk(regs[i]));
            long m3 = usedMem();
            Random r = new Random(3); int Q = Math.max(200, 4_000_000 / n); String[] q = new String[Q]; for
(int i = 0; i < Q; i++) q[i] = regs[r.nextInt(n)];
            for (int w = 0; w < 3; w++) for (String s : q) { sink += scan(arr, s) + scanL(list, s) +
(map.get(s) != null ? 1 : 0); }
            long ba = Long.MAX_VALUE, bb = Long.MAX_VALUE, bc = Long.MAX_VALUE;
            for (int round = 0; round < 5; round++) {
                long a = System.nanoTime(); for (String s : q) sink += scan(arr, s);
                long b = System.nanoTime(); for (String s : q) sink += scanL(list, s);
                long c = System.nanoTime(); for (String s : q) sink += (map.get(s) != null ? 1 : 0);
                long d = System.nanoTime();
                ba = Math.min(ba, b - a); bb = Math.min(bb, c - b); bc = Math.min(bc, d - c);
            }
            out("B11", "find ticket on exit n=" + n + " (avg us per lookup, best of 5)", String.format("array
scan %.3f | ArrayList scan %.3f | HashMap %.4f", ba / 1e3 / Q, bb / 1e3 / Q, bc / 1e3 / Q));
            out("B11", "memory for n=" + n + " tickets (KB, excl. regNo Strings)", String.format("array %.0f
| ArrayList %.0f | HashMap %.0f", (m1 - m0) / 1024.0, (m2 - m1) / 1024.0, (m3 - m2) / 1024.0));
            sink += regs.length + arr.length + list.size() + map.size();
        }
    }
    static int scan(Tk[] a, String k) { for (int i = 0; i < a.length; i++) if (a[i] != null &&
a[i].reg.equals(k)) return i; return -1; }
    static int scanL(List<Tk> a, String k) { for (int i = 0; i < a.size(); i++) if (a.get(i).reg.equals(k))
return i; return -1; }

    public static void main(String[] args) {
        out("ENV", "java", System.getProperty("java.version") + " / " +
Runtime.getRuntime().availableProcessors() + " cpu");
        b1(); b2(); b3(); b4(); b8(); b10(); b11();
    }
}
```